

# Bayesian Inference of Fat Tails in Bitcoin Returns

Using the Metropolis–Hastings Algorithm

A Project Report submitted  
for the course of  
**Probability and Statistics**

by

Jai Agarwal  
Avanish Sikarwar  
Dev Kumar  
Rishit Jain  
Abhay Tomar

Team Project Submission



Newton School of Technology

May 2026

# Declaration

We, the undersigned, declare that this report is the result of our collective academic work for the course *Probability and Statistics*. The simulations, tables, and figures in this report were generated from code and can be reproduced from the accompanying notebook. Wherever ideas, derivations, or explanations were adapted from textbooks, papers, or online documentation, they have been properly acknowledged in the references.

This report represents the complete and final version of the project submission for the course of Probability and Statistics.

Jai Agarwal (2102508753)

Avanish Sikarwar (2102508723)

Dev Kumar (2102508737)

Rishit Jain (2102508755)

Abhay Tomar (2102508702)

# Acknowledgements

We would like to express our sincere gratitude to our Mentors, Mr. Akshit Sharma, Dr. Soumya Mittal and Mr. Akshay Telmasare at Newton School of Technology, whose guidance and insightful feedback were instrumental in the development of this research. Their expertise in probability, statistics, and computational methods provided a strong foundation for this study.

We are also grateful to the open-source ecosystem that made this project possible — particularly Python, Jupyter Notebook, and the libraries NumPy, SciPy, Matplotlib, pandas, statsmodels, and yfinance for data retrieval, numerical computation, and visualisation.

We are also thankful to our fellow classmates for the stimulating discussions and collaborative environment that helped our group refine our understanding of Bayesian inference, Markov Chain Monte Carlo methods, and financial return modelling.

Finally, we are deeply grateful to our families for their constant support, patience, and encouragement throughout the duration of this project. Their belief in our academic pursuits has been a source of great motivation, and we truly appreciate the support they have provided us throughout this journey.

# Abstract

Classical financial models often assume that daily returns follow a Gaussian distribution. For Bitcoin, this assumption is too restrictive because extreme positive and negative returns occur more often than a Gaussian model predicts. This phenomenon is known as *fat tails*, and it has direct consequences for financial risk estimation.

This report develops a clean theory-to-implementation study of the problem. First, the required theory is explained using conditional probability, Bayes' theorem, Markov chains, detailed balance, the Markov Chain Monte Carlo framework, and the Metropolis–Hastings algorithm. Next, Bitcoin daily returns are modelled in four practical stages: Gaussian benchmark, classical Student- $t$  fit, Metropolis–Hastings implementation, and Metropolis–Hastings results. Finally, the models are compared using fitted graphs, log-likelihood, and Value at Risk.

Using 1,826 BTC-USD daily returns from 2 January 2020 to 31 December 2024, the Gaussian benchmark gives a log-likelihood of 3608.51 and a 1% daily Value at Risk of  $-7.61\%$  on 1,826 observations. The classical Student- $t$  fit improves the log-likelihood to 3828.53 and yields a 1% daily VaR of  $-10.02\%$ . The Bayesian Student- $t$  model estimated by Metropolis–Hastings gives a log-likelihood of 3828.50 and a 1% daily VaR of  $-9.90\%$ . These results show that Bitcoin returns are clearly fat-tailed and that both Student- $t$  models capture risk far better than the Gaussian baseline.

# Contents

|                                                                 |            |
|-----------------------------------------------------------------|------------|
| <b>Declaration</b>                                              | <b>i</b>   |
| <b>Acknowledgements</b>                                         | <b>ii</b>  |
| <b>Abstract</b>                                                 | <b>iii</b> |
| <b>Nomenclature and Abbreviations</b>                           | <b>xi</b>  |
| <b>1 Introduction</b>                                           | <b>1</b>   |
| 1.1 Background and Motivation . . . . .                         | 1          |
| 1.2 Classical and Bayesian Approaches . . . . .                 | 1          |
| 1.3 Objectives of the Report . . . . .                          | 2          |
| 1.4 Structure of the Report . . . . .                           | 2          |
| <b>2 Mathematical Background</b>                                | <b>3</b>   |
| 2.1 Probability, Events, and Random Variables . . . . .         | 3          |
| 2.2 Conditional Probability and Why It Matters . . . . .        | 3          |
| 2.3 Bayes' Theorem . . . . .                                    | 4          |
| 2.4 Expectation, Variance, and Monte Carlo Estimation . . . . . | 4          |
| 2.5 Unnormalised Densities . . . . .                            | 4          |
| 2.6 Gaussian Distribution . . . . .                             | 5          |
| 2.7 Student- $t$ Distribution and Tail Thickness . . . . .      | 5          |
| 2.8 Fat Tails and Kurtosis . . . . .                            | 5          |
| 2.9 Returns and Their Construction . . . . .                    | 6          |
| 2.10 Likelihood and Log-Likelihood . . . . .                    | 6          |
| 2.11 Value at Risk . . . . .                                    | 6          |
| <b>3 Markov Chains</b>                                          | <b>7</b>   |
| 3.1 Intuition Before Formalism . . . . .                        | 7          |
| 3.2 The Markov Property . . . . .                               | 7          |
| 3.3 Transition Kernels . . . . .                                | 7          |
| 3.4 Stationary Distribution . . . . .                           | 7          |
| 3.5 Detailed Balance and Reversibility . . . . .                | 8          |
| 3.6 Irreducibility, Aperiodicity, and Ergodicity . . . . .      | 8          |

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| 3.7      | A Simple Discrete Example . . . . .                                 | 8         |
| <b>4</b> | <b>The MCMC Framework</b>                                           | <b>9</b>  |
| 4.1      | Why Direct Sampling Can Be Difficult . . . . .                      | 9         |
| 4.2      | The Main MCMC Insight . . . . .                                     | 9         |
| 4.3      | Burn-in, Mixing, and Convergence . . . . .                          | 9         |
| <b>5</b> | <b>The Metropolis–Hastings Algorithm</b>                            | <b>11</b> |
| 5.1      | Intuition Behind the Algorithm . . . . .                            | 11        |
| 5.2      | Formal Proposal-Accept-Reject Scheme . . . . .                      | 11        |
| 5.3      | Why the Normalising Constant Cancels . . . . .                      | 11        |
| 5.4      | Symmetric Gaussian Random-Walk Proposals . . . . .                  | 12        |
| 5.5      | Proposal Width and the Exploration Trade-Off . . . . .              | 12        |
| 5.6      | Working in Log Space . . . . .                                      | 12        |
| <b>6</b> | <b>Financial Background and Model Choice</b>                        | <b>13</b> |
| 6.1      | Why Gaussian Return Models Are Often Inadequate . . . . .           | 13        |
| 6.2      | Stylized Facts of Financial Returns . . . . .                       | 13        |
| 6.3      | Why the Student- $t$ Distribution Is a Natural Compromise . . . . . | 13        |
| 6.4      | Intuition for the Three-Stage Modelling Strategy . . . . .          | 14        |
| <b>7</b> | <b>Data and Preprocessing</b>                                       | <b>15</b> |
| 7.1      | Data Source and Motivation . . . . .                                | 15        |
| 7.2      | Cell 1 — Library Imports . . . . .                                  | 15        |
| 7.3      | Cell 2 — Downloading and Inspecting the Raw Price Series . . . . .  | 16        |
| 7.4      | Cell 3 — Dataset Length . . . . .                                   | 16        |
| 7.5      | Cell 4 — Return Construction and Cleaning . . . . .                 | 17        |
| 7.6      | Cell 5 — Descriptive Statistics . . . . .                           | 17        |
| 7.7      | Cell 6 — Initial Distribution View . . . . .                        | 18        |
| <b>8</b> | <b>Gaussian Benchmark</b>                                           | <b>19</b> |
| 8.1      | Why Start with a Gaussian Model? . . . . .                          | 19        |
| 8.2      | Cell 7 — Fitting and Visualising the Gaussian Model . . . . .       | 19        |
| 8.3      | Cell 8 — Gaussian Q-Q Plot . . . . .                                | 21        |
| 8.4      | Cell 9 — Gaussian Log-Likelihood . . . . .                          | 22        |
| 8.5      | Gaussian Numerical Summary . . . . .                                | 22        |
| <b>9</b> | <b>Classical Student-<math>t</math> Benchmark</b>                   | <b>23</b> |
| 9.1      | Why the Student- $t$ Model Is the Natural Next Step . . . . .       | 23        |
| 9.2      | Cell 10 — Fitting the Classical Student- $t$ Model . . . . .        | 23        |
| 9.3      | Cell 11 — Student- $t$ Density Overlay and Comparison . . . . .     | 24        |

|           |                                                                     |           |
|-----------|---------------------------------------------------------------------|-----------|
| 9.4       | Cell 12 — Student- $t$ Q-Q Plot . . . . .                           | 25        |
| 9.5       | Cell 13 — Classical Student- $t$ Log-Likelihood . . . . .           | 25        |
| 9.6       | Interpretation and Numerical Comparison . . . . .                   | 26        |
| <b>10</b> | <b>Bayesian Formulation</b>                                         | <b>27</b> |
| 10.1      | Why Bayesian Analysis Adds Value Here . . . . .                     | 27        |
| 10.2      | Posterior for the One-Parameter Stage . . . . .                     | 27        |
| 10.3      | Posterior for the Full Three-Parameter Stage . . . . .              | 27        |
| 10.4      | Why the Two-Stage Design Helps . . . . .                            | 28        |
| <b>11</b> | <b>Metropolis–Hastings Implementation</b>                           | <b>29</b> |
| 11.1      | Stage 1: Inferring $\nu$ Alone . . . . .                            | 29        |
| 11.1.1    | Cell 14 — Stage 1 MH Sampler for $\nu$ . . . . .                    | 30        |
| 11.2      | Stage 2: Full Bayesian Inference for $(\mu, \sigma, \nu)$ . . . . . | 31        |
| 11.2.1    | Cell 15 — Stage 2 Setup: Log-Posterior and Initial Values . . . . . | 31        |
| 11.2.2    | Cell 16 — Stage 2 Sampling Loop . . . . .                           | 32        |
| 11.2.3    | Cell 17 — Burn-in Removal . . . . .                                 | 33        |
| <b>12</b> | <b>Diagnostics and Posterior Analysis</b>                           | <b>34</b> |
| 12.1      | Why Diagnostics Must Not Be Skipped . . . . .                       | 34        |
| 12.2      | Stage 1 Diagnostics . . . . .                                       | 35        |
| 12.2.1    | Cell 18 — Trace Plot for $\nu$ (Stage 1) . . . . .                  | 35        |
| 12.2.2    | Cell 19 — Posterior Histogram for $\nu$ (Stage 1) . . . . .         | 36        |
| 12.2.3    | Cell 20 — Burn-in Removal and Posterior Mean (Stage 1) . . . . .    | 36        |
| 12.2.4    | Cell 21 — Autocorrelation Plot (Stage 1) . . . . .                  | 37        |
| 12.3      | Stage 2 Diagnostics . . . . .                                       | 38        |
| 12.3.1    | Cell 22 — Trace Plots for $(\mu, \sigma, \nu)$ . . . . .            | 38        |
| 12.3.2    | Cell 23 — Posterior Histograms for $(\mu, \sigma, \nu)$ . . . . .   | 39        |
| 12.3.3    | Cell 24 — Autocorrelation Plots for Joint Chain . . . . .           | 40        |
| 12.3.4    | Cell 25 — Posterior Means . . . . .                                 | 41        |
| 12.4      | Acceptance Rate, Burn-in, and Mixing Summary . . . . .              | 41        |
| 12.5      | Posterior Summary Table . . . . .                                   | 42        |
| <b>13</b> | <b>Final Comparison of Models</b>                                   | <b>43</b> |
| 13.1      | Computing the Bayesian Student- $t$ Density . . . . .               | 43        |
| 13.1.1    | Cell 26 — Bayesian Predictive Density . . . . .                     | 43        |
| 13.2      | Cell 27 — Combined Density Plot . . . . .                           | 44        |
| 13.3      | Cell 28 — Bayesian Q-Q Plot . . . . .                               | 45        |
| 13.4      | Cells 29–31 — Log-Likelihood and Value at Risk . . . . .            | 47        |
| 13.5      | Log-Likelihood Comparison . . . . .                                 | 48        |

---

|                                                               |           |
|---------------------------------------------------------------|-----------|
| 13.6 Value at Risk Comparison . . . . .                       | 48        |
| 13.7 What Bayesian MH Adds Beyond the Classical Fit . . . . . | 48        |
| <b>14 Limitations and Extensions</b>                          | <b>50</b> |
| 14.1 Limitations of the Present Model . . . . .               | 50        |
| 14.2 Possible Extensions . . . . .                            | 50        |
| 14.3 Relation to Modern MCMC Methods . . . . .                | 50        |
| <b>15 Conclusion</b>                                          | <b>52</b> |
| <b>References</b>                                             | <b>53</b> |



# List of Figures

|      |                                                                                                                                                                                                                                                                              |    |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 7.1  | Histogram of BTC-USD daily returns used throughout the report. The extremely sharp centre and visible mass in the far tails are the visual signatures of a fat-tailed distribution. . . . .                                                                                  | 18 |
| 8.1  | Bitcoin return histogram with fitted Gaussian density. The Gaussian curve is too broad in the centre and assigns too little mass to the extreme tails. . . . .                                                                                                               | 20 |
| 8.2  | Gaussian Q-Q plot. If the Gaussian model were appropriate, the empirical quantiles would closely follow the reference line. The pronounced S-shaped curvature in both tails indicates that the data are significantly heavier-tailed than a normal distribution. . . . .     | 21 |
| 9.1  | Classical Student- $t$ fit (green) compared with the Gaussian benchmark (red) and empirical histogram. The Student- $t$ curve matches the sharp central peak and the heavy tails far better than the Gaussian. . . . .                                                       | 24 |
| 9.2  | Student- $t$ Q-Q plot. The empirical quantiles now track the reference line much more closely than in the Gaussian case (Figure 8.2), confirming that the Student- $t$ family captures the tail behaviour of Bitcoin returns. . . . .                                        | 25 |
| 12.1 | Trace plot for the stage-1 $\nu$ chain (5000 iterations). The chain moves rapidly away from the starting value of 5 and fluctuates around a stable region near $\nu \approx 2.6$ for the remainder of the run, with no persistent drift. This indicates good mixing. . . . . | 35 |
| 12.2 | Posterior histogram for $\nu$ after stage-1 sampling. The distribution concentrates between 2.4 and 3.2, with mass centred near 2.6. The narrow posterior confirms that the data are informative about tail heaviness. . . . .                                               | 36 |
| 12.3 | Autocorrelation plot for the stage-1 chain after burn-in. Significant autocorrelation decays to near zero by approximately lag 15–20, which is typical for a random-walk Metropolis sampler. The chain mixes reasonably well for a basic implementation. . . . .             | 37 |
| 12.4 | Trace plots for the joint stage-2 MH chains of $\mu$ , $\sigma$ , and $\nu$ after burn-in removal. Each parameter fluctuates around a stable level rather than drifting over time, confirming that the chain has converged to its stationary distribution. . . . .           | 38 |

|      |                                                                                                                                                                                                                                                                                                                                                                                                                          |    |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 12.5 | Posterior histograms for the three Student- $t$ parameters from stage-2 sampling. All three posteriors are unimodal and approximately symmetric, indicating that the chain has explored the posterior without getting trapped. . . . .                                                                                                                                                                                   | 39 |
| 12.6 | Autocorrelation plots for the joint post-burn-in chains of $\mu$ , $\sigma$ , and $\nu$ . $\sigma$ and $\nu$ mix well with autocorrelation decaying to near zero by lag 15. The $\mu$ chain shows somewhat higher autocorrelation, which is common for location parameters in heavy-tailed models. All chains are usable for posterior inference. . . . .                                                                | 40 |
| 13.1 | Final density comparison: Gaussian (red), classical Student- $t$ (green), and Bayesian MH Student- $t$ (black dashed). The Gaussian fit is too thin in the tails and too broad in the centre, while both Student- $t$ fits track the empirical shape. The classical and Bayesian Student- $t$ densities are nearly indistinguishable, confirming that the posterior means closely match the MLE point estimates. . . . . | 45 |
| 13.2 | Bayesian MH Student- $t$ Q-Q plot. The fit is consistent with the data for the bulk of the distribution. The small number of extreme outliers that deviate from the line in the far tails reflect individual extreme Bitcoin price events that are difficult to capture with any stationary parametric model. . . . .                                                                                                    | 46 |

# List of Tables

|      |                                                                                        |    |
|------|----------------------------------------------------------------------------------------|----|
| 7.1  | First five rows of the downloaded BTC-USD dataset (Close column shown in USD). . . . . | 16 |
| 7.2  | First five rows after adding the Return column (2 January 2020 onward). . . . .        | 17 |
| 7.3  | Descriptive statistics of BTC-USD daily returns used throughout the report. . . . .    | 17 |
| 12.1 | Key posterior summaries from the Metropolis–Hastings runs. . . . .                     | 42 |
| 13.1 | Log-likelihood comparison across the three fitted models. . . . .                      | 48 |
| 13.2 | 1% daily Value at Risk under the three fitted models. . . . .                          | 48 |

# Nomenclature and Abbreviations

| Symbol / Abbreviation     | Meaning                                                 |
|---------------------------|---------------------------------------------------------|
| <b>MCMC</b>               | Markov Chain Monte Carlo                                |
| <b>MH</b>                 | Metropolis–Hastings                                     |
| <b>MLE</b>                | Maximum Likelihood Estimation                           |
| <b>VaR</b>                | Value at Risk                                           |
| <b>BTC</b>                | Bitcoin                                                 |
| $\pi(\theta)$             | Target or posterior distribution                        |
| $q(\theta'   \theta)$     | Proposal distribution                                   |
| $\alpha(\theta, \theta')$ | Acceptance probability                                  |
| $\mu$                     | Location parameter of the return distribution           |
| $\sigma$                  | Scale parameter of the return distribution              |
| $\nu$                     | Degrees-of-freedom parameter controlling tail heaviness |
| <b>ACF</b>                | Autocorrelation Function                                |
| <b>ESS</b>                | Effective Sample Size                                   |

# Chapter 1

## Introduction

### 1.1 Background and Motivation

Any statistical model for financial returns is ultimately judged by how honestly it represents risk. If a model understates the probability of rare but severe losses, then the numerical outputs may look precise while still being misleading in practice. This is why tail behaviour matters so much in finance: the extreme events are infrequent, but they are often the events that matter most.

The Gaussian distribution is the default benchmark in many introductory models because it is mathematically convenient and has a long history in applied statistics. But real financial returns rarely behave like ideal Gaussian data. They tend to show outliers, asymmetry, periods of calm followed by turbulence, and—most importantly—more probability mass in the tails than a normal curve allows. In other words, extreme returns happen too often for the Gaussian model to be fully trusted.

Bitcoin is a natural case study for this issue. It trades continuously, reacts strongly to news and market sentiment, and frequently exhibits large daily movements. That makes it a useful dataset for studying heavy-tailed modelling in a setting where Gaussian assumptions are particularly fragile.

### 1.2 Classical and Bayesian Approaches

There are two closely related questions in this project. The first is *descriptive*: which distribution fits Bitcoin returns better? The second is *inferential*: once a heavy-tailed model is chosen, how uncertain are we about its parameters?

The classical Student- $t$  fit addresses the first question using maximum likelihood estimation and produces a single best-fitting parameter triple  $(\nu, \mu, \sigma)$ . The Bayesian Metropolis–Hastings approach addresses the second question by producing a *posterior distribution* over parameters instead of a single point. This is valuable because in real data analysis the parameters themselves are uncertain, especially when the main object

of interest is tail heaviness.

### 1.3 Objectives of the Report

1. Explain the mathematical and probabilistic theory behind Metropolis–Hastings with enough intuition that the algorithm does not feel like a black box.
2. Explain why Gaussian models fail for Bitcoin returns and why the Student- $t$  family is a natural alternative.
3. Present the notebook material in a clean academic sequence: Gaussian work, classical Student- $t$  work, MH implementation, MH results, and final comparison.
4. Interpret the numerical results in terms of financial risk, especially log-likelihood and 1% daily Value at Risk.

### 1.4 Structure of the Report

The report develops theory carefully before turning to the empirical application. This is deliberate: in a strong academic report, the implementation should appear as a consequence of the theory, not as an isolated block of code. The reader should understand what is being sampled, why direct sampling is difficult, and why the Metropolis–Hastings construction solves that difficulty.

Chapters 2–6 develop all necessary theory. Chapters 7–13 present the empirical application with every notebook cell shown in context alongside its output.

## Chapter 2

# Mathematical Background

## 2.1 Probability, Events, and Random Variables

Probability theory begins with a sample space, events, and a probability measure. In this report, the random object of interest is the daily return of Bitcoin. A return is a random variable because its value is not known in advance and changes from day to day.

For a continuous random variable  $X$  with density  $f(x)$ , probabilities are computed through integration:

$$P(a \leq X \leq b) = \int_a^b f(x) dx, \quad \int_{-\infty}^{\infty} f(x) dx = 1. \quad (2.1)$$

The density does not give probability at a single point; it describes how probability mass is distributed across intervals.

## 2.2 Conditional Probability and Why It Matters

Conditional probability is defined by

$$P(A | B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0. \quad (2.2)$$

This is one of the conceptual foundations of Bayesian inference. Once event  $B$  is known to have occurred, the probability of  $A$  is updated relative to that information. In this report, the *data* play the role of the information that becomes known. We start with uncertainty about the parameters of the return distribution and then update that uncertainty after observing actual Bitcoin returns.

## 2.3 Bayes' Theorem

Bayes' theorem turns conditional probability into a full updating rule:

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}. \quad (2.3)$$

When the unknown quantity is a parameter vector  $\theta$  and the observed data are  $\mathbf{r} = (r_1, \dots, r_n)$ , this becomes

$$p(\theta | \mathbf{r}) = \frac{p(\mathbf{r} | \theta) p(\theta)}{p(\mathbf{r})}. \quad (2.4)$$

The four terms have clear meanings: (i)  $p(\theta)$  is the *prior*, representing beliefs before the data are observed; (ii)  $p(\mathbf{r} | \theta)$  is the *likelihood*; (iii)  $p(\theta | \mathbf{r})$  is the *posterior*, representing updated belief; (iv)  $p(\mathbf{r})$  is the normalising constant. The posterior is the central target of Bayesian inference. The normalising constant  $p(\mathbf{r})$  usually involves an integral that is hard to compute—this is one of the main reasons why MCMC methods are needed.

## 2.4 Expectation, Variance, and Monte Carlo Estimation

If  $X$  has density  $f(x)$ , then the expectation of a function  $g(X)$  is

$$\mathbb{E}[g(X)] = \int g(x) f(x) dx. \quad (2.5)$$

Monte Carlo methods approximate such integrals using samples:

$$\mathbb{E}[g(X)] \approx \frac{1}{N} \sum_{i=1}^N g(x_i). \quad (2.6)$$

This is the final purpose of MCMC: to approximate expectations, posterior means, tail probabilities, and predictive quantities by averaging over samples.

## 2.5 Unnormalised Densities

Many real problems only provide a density up to a constant:

$$\pi(\theta) = \frac{\tilde{\pi}(\theta)}{Z}, \quad Z = \int \tilde{\pi}(\theta) d\theta. \quad (2.7)$$



In Bayesian statistics the posterior often has a likelihood-times-prior form that is easy to evaluate pointwise but hard to normalise globally. Metropolis–Hastings works with *ratios* of unnormalised densities, so  $Z$  cancels—this is its central computational advantage.

## 2.6 Gaussian Distribution

If  $X \sim \mathcal{N}(\mu, \sigma^2)$ , then

$$f(x \mid \mu, \sigma) = \frac{1}{\sqrt{2\pi} \sigma} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right). \quad (2.8)$$

The Gaussian distribution is symmetric, light-tailed, and completely described by its mean and variance. Its mathematical tractability is the reason it is the first model tried in practice. However, that convenience becomes a weakness when the observed data contain many large deviations from the mean.

## 2.7 Student- $t$ Distribution and Tail Thickness

The Student- $t$  family introduces one additional parameter, the degrees of freedom  $\nu$ , to control tail behaviour:

$$f(x \mid \nu, \mu, \sigma) = \frac{\Gamma(\frac{\nu+1}{2})}{\Gamma(\frac{\nu}{2}) \sqrt{\nu\pi} \sigma} \left(1 + \frac{1}{\nu} \left(\frac{x - \mu}{\sigma}\right)^2\right)^{-\frac{\nu+1}{2}}. \quad (2.9)$$

The parameter  $\nu$  controls tail heaviness: when  $\nu$  is small, very large returns remain plausible; when  $\nu \rightarrow \infty$  the density converges to the Gaussian. The Student- $t$  is therefore a strict generalisation of the Gaussian benchmark. Key moment conditions: the mean exists for  $\nu > 1$ ; the variance for  $\nu > 2$ ; the kurtosis for  $\nu > 4$ .

## 2.8 Fat Tails and Kurtosis

“Fat tails” means that extreme outcomes are more likely than under a Gaussian model. In a histogram this appears as a sharp central peak together with more mass in the far left and far right tails. In a Q-Q plot against the Gaussian it typically appears as systematic curvature at the extremes. The excess kurtosis for a Student- $t$  with  $\nu > 4$  is

$$\kappa = \frac{6}{\nu - 4}. \quad (2.10)$$

As  $\nu$  decreases toward 4 from above, the kurtosis rises rapidly, making  $\nu$  a meaningful tail-risk indicator.

## 2.9 Returns and Their Construction

If  $P_t$  denotes the closing price on day  $t$ , then the simple daily return is

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}. \quad (2.11)$$

A value of  $r_t = -0.10$  corresponds to a 10% loss in one day.

## 2.10 Likelihood and Log-Likelihood

For observations  $r_1, \dots, r_n$  and parameter vector  $\theta$ ,

$$L(\theta) = \prod_{i=1}^n f(r_i | \theta), \quad \ell(\theta) = \sum_{i=1}^n \log f(r_i | \theta). \quad (2.12)$$

The log-likelihood is used in practice because it turns products into sums and improves numerical stability.

## 2.11 Value at Risk

The 1% daily Value at Risk is defined by

$$\text{VaR}_{0.01} = F^{-1}(0.01), \quad (2.13)$$

where  $F$  is the fitted return CDF. If the 1% VaR is  $-0.10$ , then roughly one day in a hundred is expected to have a loss worse than 10%. VaR translates the abstract issue of fat tails into a directly interpretable risk metric.

## Chapter 3

# Markov Chains

### 3.1 Intuition Before Formalism

Picture a Markov chain as a rule for moving from one state to another. If the state is the current value of a parameter, then the chain is a recipe that tells us how to generate the next candidate parameter value from the current one. The special feature is that the next move depends only on where the chain is *now*, not on the entire history of how it got there.

### 3.2 The Markov Property

A stochastic process  $\{X_t\}_{t \geq 0}$  is Markov if

$$P(X_{t+1} \mid X_0, X_1, \dots, X_t) = P(X_{t+1} \mid X_t). \quad (3.1)$$

Once the present state is known, the past adds no extra information for predicting the next state. For MCMC, this is exactly what is needed: at iteration  $t$ , the sampler only needs the current parameter value and the proposal rule.

### 3.3 Transition Kernels

In continuous state spaces, the chain is described by a transition kernel  $T(x' \mid x)$ : the probability density of moving to state  $x'$  given current state  $x$ . The design problem of MCMC is a kernel-design problem—how to build  $T$  so that repeated application eventually produces samples from the desired target distribution  $\pi$ .

### 3.4 Stationary Distribution

A distribution  $\pi$  is stationary for a chain with kernel  $T$  if

$$\pi(x') = \int \pi(x) T(x' \mid x) dx. \quad (3.2)$$

That is, if the current state were already distributed according to  $\pi$ , one more transition would leave the distribution unchanged.

### 3.5 Detailed Balance and Reversibility

A stronger but easier sufficient condition for stationarity is *detailed balance*:

$$\pi(x) T(x' | x) = \pi(x') T(x | x'). \quad (3.3)$$

Integrating both sides over  $x$  and noting that  $\int T(x | x') dx = 1$  recovers exactly (3.2). Detailed balance is the central design principle of Metropolis–Hastings: the acceptance probability is chosen specifically so that the resulting transition kernel satisfies this identity.

### 3.6 Irreducibility, Aperiodicity, and Ergodicity

For a chain to be practically useful, it must be irreducible (able to reach all relevant regions of the state space) and aperiodic (not trapped in deterministic cycles). When these conditions hold together with stationarity, the ergodic theorem guarantees

$$\frac{1}{N} \sum_{t=1}^N g(X_t) \xrightarrow{N \rightarrow \infty} \mathbb{E}_\pi[g(X)], \quad (3.4)$$

which is the formal statement that justifies using a Markov chain to approximate posterior expectations.

### 3.7 A Simple Discrete Example

Consider a two-state chain with transition matrix

$$T = \begin{pmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{pmatrix}. \quad (3.5)$$

Solving  $\pi T = \pi$  with  $\pi_0 + \pi_1 = 1$  gives  $\pi_0 = 4/7$  and  $\pi_1 = 3/7$ . The long-run proportion of time spent in each state stabilises at these values, regardless of where the chain starts—exactly the idea that MCMC exploits in continuous spaces.

## Chapter 4

# The MCMC Framework

### 4.1 Why Direct Sampling Can Be Difficult

Suppose we want to sample from a posterior  $\pi(\theta \mid \mathbf{r})$ . In a simple textbook problem we may have a conjugate posterior that admits direct simulation. But in realistic Bayesian problems the posterior is often known only up to a constant:

$$\pi(\theta \mid \mathbf{r}) \propto p(\mathbf{r} \mid \theta) p(\theta). \quad (4.1)$$

The denominator  $p(\mathbf{r}) = \int p(\mathbf{r} \mid \vartheta) p(\vartheta) d\vartheta$  is easy to write but hard to compute numerically, especially in higher dimensions where grid and rejection methods also break down (grid requirements grow exponentially, and rejection sampling requires a dominating proposal).

### 4.2 The Main MCMC Insight

MCMC avoids these problems by changing the task. Instead of sampling directly from  $\pi$ , it constructs a Markov chain whose stationary distribution *is*  $\pi$ . The chain does not need the normalising constant—it only needs the target up to proportionality because the acceptance rule depends on ratios.

*We do not normalise first and sample second. We design a stochastic process that automatically spends the right amount of time in each region of parameter space.*

### 4.3 Burn-in, Mixing, and Convergence

Three concepts appear throughout any MCMC report:

- **Burn-in:** the initial transient period before the chain settles near the typical region of the target.
- **Mixing:** how rapidly the chain moves through the parameter space.

- **Convergence:** whether the distribution of the chain has become close to the target.

These three ideas are not independent concerns but aspects of the same practical question: how long must the chain run before its output is trustworthy? In the Metropolis–Hastings implementation presented in Chapter 11, burn-in is handled by discarding the opening iterations before computing any summaries; mixing is monitored through trace plots and autocorrelation functions; and convergence is assessed by verifying that the chain settles to a stable region with no persistent trend. Chapter 12 demonstrates each of these checks on the Bitcoin return chains.

## Chapter 5

# The Metropolis–Hastings Algorithm

### 5.1 Intuition Behind the Algorithm

Imagine exploring an unknown landscape in the dark, where elevation corresponds to posterior density. You stand at one point; the proposal rule suggests a nearby point. If the new point is higher, moving there is sensible. If it is lower, moving there should still be possible *sometimes*—otherwise the chain could never explore the full space. Metropolis–Hastings captures exactly this logic, balancing preference for high-probability regions against continued exploration of the full space.

### 5.2 Formal Proposal-Accept-Reject Scheme

At iteration  $t$ , with current state  $\theta_t$ :

1. Draw a proposal  $\theta' \sim q(\theta' \mid \theta_t)$ .
2. Compute the acceptance ratio

$$r(\theta_t, \theta') = \frac{\pi(\theta') q(\theta_t \mid \theta')}{\pi(\theta_t) q(\theta' \mid \theta_t)}. \quad (5.1)$$

3. Accept with probability  $\alpha(\theta_t, \theta') = \min(1, r)$ .
4. Set  $\theta_{t+1} = \theta'$  if accepted; otherwise  $\theta_{t+1} = \theta_t$ .

The rule  $\alpha = \min(1, r)$  is not arbitrary—it is chosen so that the resulting transition kernel satisfies detailed balance (3.3).

### 5.3 Why the Normalising Constant Cancels

If  $\pi(\theta) = \tilde{\pi}(\theta)/Z$ , then

$$r(\theta, \theta') = \frac{\tilde{\pi}(\theta') q(\theta \mid \theta')}{\tilde{\pi}(\theta) q(\theta' \mid \theta)}. \quad (5.2)$$

The unknown  $Z$  disappears. This is one of the most important ideas in the report: Metropolis–Hastings can sample from posteriors whose normalising constants are computationally intractable.

## 5.4 Symmetric Gaussian Random-Walk Proposals

In this project, proposals are symmetric Gaussian random walks:

$$\theta' = \theta + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \tau^2). \quad (5.3)$$

Because  $q(\theta' \mid \theta) = q(\theta \mid \theta')$ , the ratio simplifies to  $r = \pi(\theta')/\pi(\theta)$ —the standard Metropolis special case.

## 5.5 Proposal Width and the Exploration Trade-Off

The proposal width  $\tau$  largely determines chain behaviour:

- **Too small:** most proposals are accepted but the chain moves in tiny steps and remains highly autocorrelated.
- **Too large:** most proposals land in low-probability regions and are rejected, causing the chain to get stuck.
- **Well-tuned:** meaningful movement with an acceptance rate in the commonly recommended range of 20–40%.

## 5.6 Working in Log Space

Posterior densities can be extremely small when they are products over many observations. The implementation works with log-densities:

$$\log r = \log \tilde{\pi}(\theta') - \log \tilde{\pi}(\theta), \quad (5.4)$$

and the decision rule becomes  $\log u < \log r$  for  $u \sim \text{Uniform}(0, 1)$ . This avoids numerical underflow and is standard practice in MCMC coding.



## Chapter 6

# Financial Background and Model Choice

## 6.1 Why Gaussian Return Models Are Often Inadequate

The Gaussian model assigns very small probability to extreme observations, which is useful when returns are generated by many small additive shocks with no major jumps. But financial returns often exhibit large moves driven by policy changes, exchange failures, macroeconomic news, or abrupt shifts in market sentiment. If such extremes are treated as effectively impossible, risk is understated—a technical flaw with real consequences in capital allocation and risk management.

For Bitcoin specifically, this issue is acute. The asset trades continuously, reacts strongly to regulatory announcements and market sentiment, and has historically shown daily swings that would be essentially impossible under a Gaussian model.

## 6.2 Stylized Facts of Financial Returns

Empirical return series often exhibit: heavy tails relative to the Gaussian benchmark; central peaks sharper than the normal density; volatility clustering; and occasional outliers from jumps. This report focuses on the most visible distributional issue: fat tails. Time-varying volatility (GARCH, stochastic volatility) is left as a natural extension.

## 6.3 Why the Student- $t$ Distribution Is a Natural Compromise

The Student- $t$  family is attractive because it is only one step more complicated than the Gaussian yet much more flexible in the tails. It retains a simple location-scale form while allowing  $\nu$  to adapt to the data. It is rich enough to model heavy-tailed returns but simple enough to remain interpretable—an ideal balance for an educational project that connects probabilistic theory, estimation, and financial interpretation.

## 6.4 Intuition for the Three-Stage Modelling Strategy

The project is deliberately structured to build intuition in layers:

1. **Start with Gaussian** — because it is the natural first guess. Most introductory statistics courses tell us that averages and sums of random variables tend toward Gaussian distributions (Central Limit Theorem). It is therefore perfectly sensible to first ask: “could returns simply be normal?” The answer turns out to be no, but the Gaussian fit quantifies *exactly how much* it fails, which motivates the next step.
2. **Classical Student- $t$**  — because it adds one extra parameter  $\nu$  that lets the tails be as heavy as the data require. Maximum likelihood estimation gives a single best-fitting  $(\nu, \mu, \sigma)$ . This answers the descriptive question definitively.
3. **Bayesian MH** — because the classical fit gives a point estimate, but  $\nu$  is inherently uncertain. Metropolis–Hastings produces a *distribution* over plausible values of  $\nu$  (and  $\mu, \sigma$ ), quantifying that uncertainty. This is the key conceptual payoff: a range of good parameters rather than a single answer, which is more honest and more useful for risk management.

The following chapters implement this three-stage strategy in order, beginning with the data acquisition and preprocessing in Chapter 7.

## Chapter 7

# Data and Preprocessing

## 7.1 Data Source and Motivation

The empirical dataset consists of BTC-USD daily closing prices downloaded from Yahoo Finance for the period 1 January 2020 to 1 January 2025. Bitcoin is a convenient asset because it trades every day and exhibits substantial variability, which makes tail behaviour easier to study than in calmer asset classes.

## 7.2 Cell 1 — Library Imports

All external libraries are imported in a single cell at the top of the notebook, a standard practice that makes dependencies explicit and reproducible.

```
1 import yfinance as yf
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 from scipy.stats import norm
6 from scipy import stats
7 from scipy.stats import t
8 from statsmodels.graphics.tsaplots import plot_acf
```

Listing 7.1: Library imports.

Each library plays a specific role: `yfinance` retrieves price data from Yahoo Finance; `pandas` handles the data frame operations; `numpy` provides the core numerical array infrastructure; `matplotlib` produces all figures; `scipy.stats` supplies the Gaussian and Student-*t* distribution functions; and `statsmodels` provides the autocorrelation plot for MCMC diagnostics.

## 7.3 Cell 2 — Downloading and Inspecting the Raw Price Series

The closing prices are downloaded for the five-year window. The `head()` call confirms that the data has loaded correctly and shows the multi-level column structure that `yfinance` returns.

```
1 btc = yf.download("BTC-USD", start="2020-01-01", end="2025-01-01")
2 btc.head()
```

Listing 7.2: Download BTC-USD price data and inspect the first rows.

Table 7.1: First five rows of the downloaded BTC-USD dataset (Close column shown in USD).

| Date       | Close   | High    | Low     | Open    | Volume         |
|------------|---------|---------|---------|---------|----------------|
| 2020-01-01 | 7200.17 | 7254.33 | 7174.94 | 7194.89 | 18,565,664,997 |
| 2020-01-02 | 6985.47 | 7212.16 | 6935.27 | 7202.55 | 20,802,083,465 |
| 2020-01-03 | 7344.88 | 7413.72 | 6914.99 | 6984.43 | 28,111,481,032 |
| 2020-01-04 | 7410.66 | 7427.39 | 7309.51 | 7345.38 | 18,444,271,275 |
| 2020-01-05 | 7411.32 | 7544.50 | 7400.54 | 7410.45 | 19,725,074,095 |

## 7.4 Cell 3 — Dataset Length

A quick check confirms the number of rows before returns are computed.

```
1 len(btc)
```

Listing 7.3: Check the number of rows in the raw dataset.

```
1827
```

There are 1,827 daily price observations covering the five-year window.

## 7.5 Cell 4 — Return Construction and Cleaning

Simple percentage returns are computed from the closing-price series using equation (2.11). The first row (which becomes NaN after differencing) is removed with `dropna()`, leaving 1,826 usable daily return observations.

```
1 btc["Return"] = btc[("Close", "BTC-USD")].pct_change()
2 btc = btc.dropna()
3 btc.head()
```

Listing 7.4: Compute daily percentage returns and remove the first missing value.

Table 7.2: First five rows after adding the Return column (2 January 2020 onward).

| Date       | Close   | ... | Return   |
|------------|---------|-----|----------|
| 2020-01-02 | 6985.47 | ... | −0.02982 |
| 2020-01-03 | 7344.88 | ... | +0.05145 |
| 2020-01-04 | 7410.66 | ... | +0.00895 |
| 2020-01-05 | 7411.32 | ... | +0.00009 |
| 2020-01-06 | 7769.22 | ... | +0.04830 |

## 7.6 Cell 5 — Descriptive Statistics

```
1 btc["Return"].describe()
```

Listing 7.5: Summary statistics of BTC-USD daily returns.

Table 7.3: Descriptive statistics of BTC-USD daily returns used throughout the report.

| Statistic          | Value     |
|--------------------|-----------|
| Count              | 1826      |
| Mean               | 0.001978  |
| Standard deviation | 0.033546  |
| Minimum            | −0.371695 |
| 25th percentile    | −0.012894 |
| Median             | 0.000677  |
| 75th percentile    | 0.016544  |
| Maximum            | 0.187465  |

The statistics already hint that the series is nontrivial. The range between the minimum (−37.2%) and maximum (+18.7%) is very wide. A standard deviation of 3.35% per day with a minimum of −37.2% would correspond to an eleven-sigma event under a Gaussian model—essentially impossible by Gaussian assumptions but observed in the data.

## 7.7 Cell 6 — Initial Distribution View

Before fitting any parametric model it is important to inspect the empirical histogram. The shape is already suggestive: the centre is sharp, but the tails contain enough large observations that a Gaussian fit should be treated with caution.

```
1 plt.figure(figsize=(10,6))  
2 plt.hist(btc["Return"], bins=100)  
3 plt.show()
```

Listing 7.6: Raw histogram of BTC-USD daily returns.

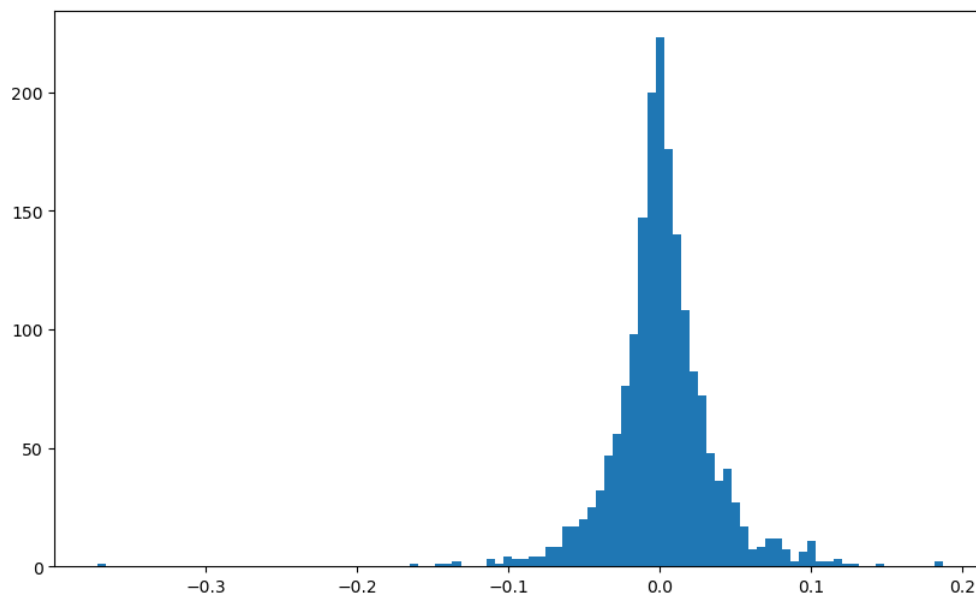


Figure 7.1: Histogram of BTC-USD daily returns used throughout the report. The extremely sharp centre and visible mass in the far tails are the visual signatures of a fat-tailed distribution.

## Chapter 8

# Gaussian Benchmark

### 8.1 Why Start with a Gaussian Model?

The first intuitive guess for return distributions is Gaussian, and for good reason. The Central Limit Theorem tells us that the sum of many small independent random shocks converges to a Gaussian distribution. If daily returns were the aggregate of many small, independent pieces of news hitting the market, we would naturally expect them to look roughly normal. Gaussian models are also mathematically convenient: they are fully characterised by just two parameters ( $\mu$  and  $\sigma$ ), estimated trivially from the sample mean and standard deviation.

The goal of this section is not to argue that returns are normal, but to *quantify exactly how and where the normal fit fails*, so that the improvement from the Student-*t* model is measurable rather than merely claimed.

### 8.2 Cell 7 — Fitting and Visualising the Gaussian Model

The code estimates  $\mu$  and  $\sigma$  from the sample, constructs the fitted density over the observed return range, and overlays it on the empirical histogram.

The code estimates  $\mu$  as the sample mean and  $\sigma$  as the sample standard deviation — the maximum likelihood estimates for the Gaussian family — then evaluates the fitted density on a fine grid spanning the observed return range. Overlaying this curve on a density-normalised histogram provides an immediate visual diagnostic: a good fit would place the curve's peak at the histogram's peak and trace the tail heights with reasonable accuracy. As Figure 8.1 shows, the Gaussian curve is simultaneously too wide in the centre and too thin in the tails, which is the characteristic failure pattern for Gaussian fits to financial return data.

```

1 mu = btc["Return"].mean()
2 sigma = btc["Return"].std()
3
4 x = np.linspace(btc["Return"].min(),
5                 btc["Return"].max(),
6                 1000)
7
8 gaussian = norm.pdf(x, mu, sigma)
9
10 plt.figure(figsize=(10,6))
11 plt.hist(btc["Return"],
12         bins=100,
13         density=True,
14         alpha=0.6,
15         label="Actual Returns")
16
17 plt.plot(x,
18         gaussian,
19         color="red",
20         linewidth=2,
21         label="Gaussian Fit")
22
23 plt.legend()
24 plt.show()

```

Listing 8.1: Gaussian parameter estimation, density overlay.

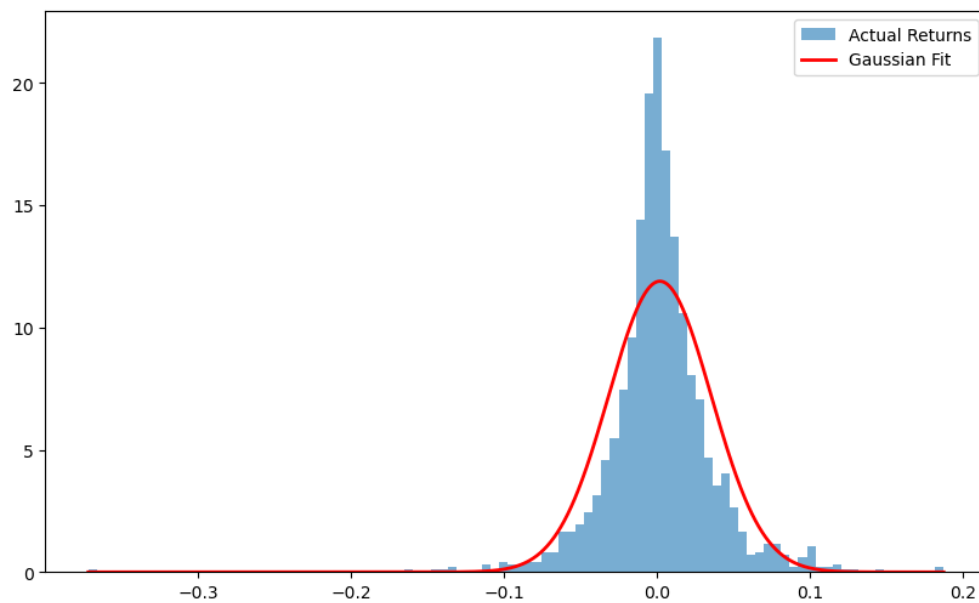


Figure 8.1: Bitcoin return histogram with fitted Gaussian density. The Gaussian curve is too broad in the centre and assigns too little mass to the extreme tails.



### 8.3 Cell 8 — Gaussian Q-Q Plot

```
1 standardized_gaussian = (  
2     (btc["Return"] - mu) / sigma  
3 )  
4 stats.probplot(  
5     standardized_gaussian,  
6     dist="norm",  
7     plot=plt  
8 )  
9  
10 plt.show()
```

Listing 8.2: Gaussian Q-Q plot for standardised returns.

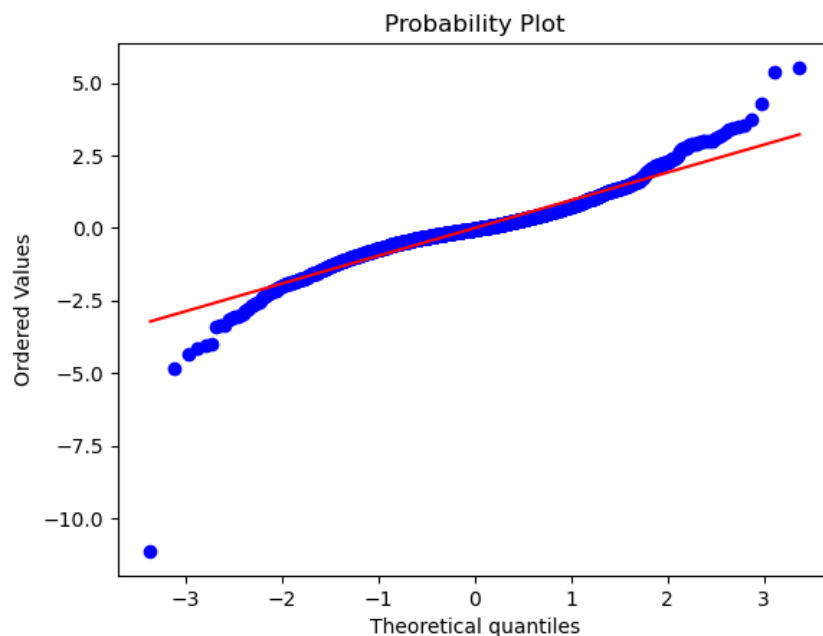


Figure 8.2: Gaussian Q-Q plot. If the Gaussian model were appropriate, the empirical quantiles would closely follow the reference line. The pronounced S-shaped curvature in both tails indicates that the data are significantly heavier-tailed than a normal distribution.

The histogram overlay shows that the Gaussian curve gives a rough description of the middle of the sample but assigns too little probability to large positive and negative returns. The Q-Q plot makes the same point more sharply: the data curve strongly away from the reference diagonal in both tails.

## 8.4 Cell 9 — Gaussian Log-Likelihood

```
1 log_likelihood_gaussian = np.sum(  
2     norm.logpdf(btc["Return"], mu, sigma)  
3 )  
4 print(log_likelihood_gaussian)
```

Listing 8.3: Compute the Gaussian log-likelihood for later comparison.

```
3608.508377370954
```

## 8.5 Gaussian Numerical Summary

For the benchmark Gaussian model, the fitted mean and standard deviation are

$$\mu = 0.001978, \quad \sigma = 0.033546$$

The mean of roughly 0.2% per day reflects the mild positive drift of Bitcoin over the 2020–2024 window, while the standard deviation of 3.35% captures the typical scale of daily movement. The associated log-likelihood is 3608.51, which provides the numerical baseline against which the heavier-tailed models will be judged in Chapter 13.

A useful sanity check on these numbers: the empirical minimum return of  $-37.2\%$  lies approximately eleven standard deviations below the fitted Gaussian mean. Under a true Gaussian model this event would have probability indistinguishable from zero, yet it appears once in 1,826 observations. This single data point is already strong evidence that the Gaussian assumption is inadequate, and it motivates the Student- $t$  analysis in the next chapter.

## Chapter 9

# Classical Student- $t$ Benchmark

## 9.1 Why the Student- $t$ Model Is the Natural Next Step

Once the Gaussian model shows clear tail failure, the most natural next step is a heavier-tailed parametric family. The Student- $t$  distribution is ideal here for a simple intuitive reason: it keeps the familiar location-scale structure while adding a single new parameter  $\nu$  that the data themselves can determine. Instead of assuming the tails decay at a fixed Gaussian rate, we let the evidence decide how fast or slow that decay should be.

Maximum likelihood estimation (MLE) of  $(\nu, \mu, \sigma)$  answers the question: “among all Student- $t$  distributions, which one fits the observed returns best?” This is a classical, frequentist answer. It is direct and numerically efficient, and it provides the baseline against which the Bayesian MH analysis will be compared.

## 9.2 Cell 10 — Fitting the Classical Student- $t$ Model

```
1 nu, mu_t, sigma_t = t.fit(btc["Return"])
2 print(nu, mu_t, sigma_t)
```

Listing 9.1: Fit a three-parameter Student- $t$  distribution by MLE and print the estimates.

```
2.605458432115676 0.0011337183369557547 0.019715919504470526
```

The estimated degrees of freedom  $\hat{\nu} \approx 2.61$  is a very small value. Recall from Section 2.7 that the variance of a Student- $t$  distribution exists only for  $\nu > 2$ , and the kurtosis only for  $\nu > 4$ . An estimate of 2.61 means that the data are pushing the fit as close to the boundary of infinite variance as the model allows—a striking quantitative statement about how heavy-tailed Bitcoin returns truly are.

### 9.3 Cell 11 — Student- $t$ Density Overlay and Comparison

```

1 student_t = t.pdf(x, nu, mu_t, sigma_t)
2 plt.figure(figsize=(10,6))
3
4 plt.hist(btc["Return"],
5         bins=100,
6         density=True,
7         alpha=0.6,
8         label="Actual Returns")
9
10 plt.plot(x,
11         gaussian,
12         color="red",
13         linewidth=2,
14         label="Gaussian Fit")
15
16 plt.plot(x,
17         student_t,
18         color="green",
19         linewidth=2,
20         label="Student-t Fit")
21
22 plt.legend()
23 plt.show()

```

Listing 9.2: Plot the classical Student- $t$  density alongside the Gaussian benchmark.

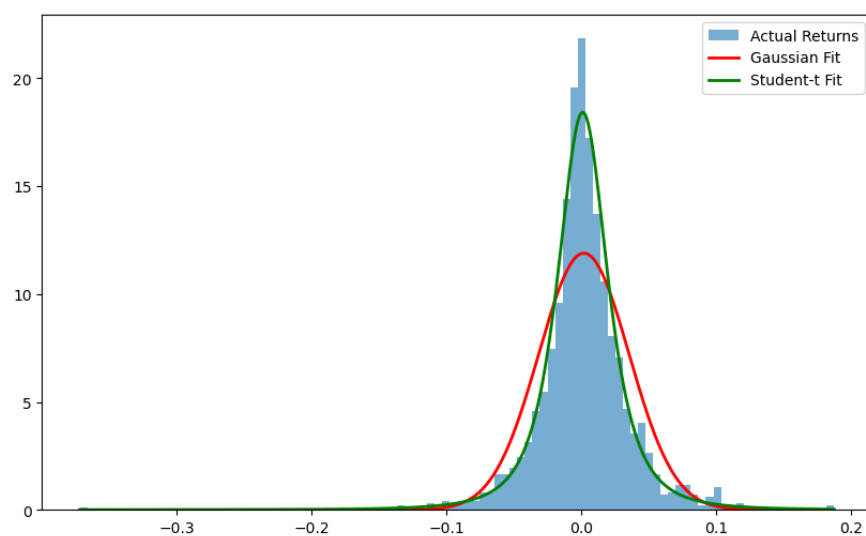


Figure 9.1: Classical Student- $t$  fit (green) compared with the Gaussian benchmark (red) and empirical histogram. The Student- $t$  curve matches the sharp central peak and the heavy tails far better than the Gaussian.

## 9.4 Cell 12 — Student- $t$ Q-Q Plot

```

1 standardized_returns = (
2     (btc["Return"] - mu_t) / sigma_t)
3 stats.probplot(
4     standardized_returns,
5     dist=t,
6     sparams=(nu,),
7     plot=plt)
8
9 plt.show()

```

Listing 9.3: Q-Q plot of standardised returns against the fitted Student- $t$  distribution.

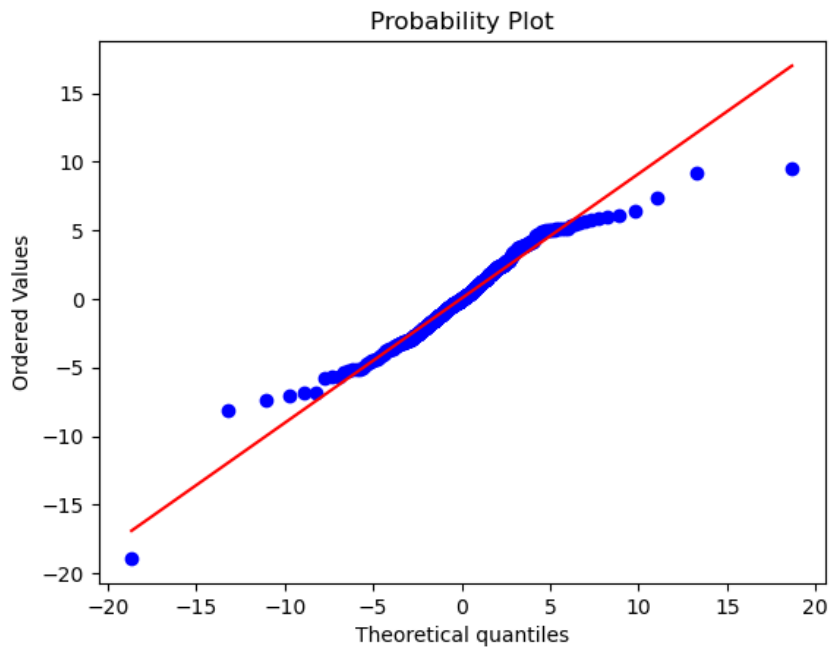


Figure 9.2: Student- $t$  Q-Q plot. The empirical quantiles now track the reference line much more closely than in the Gaussian case (Figure 8.2), confirming that the Student- $t$  family captures the tail behaviour of Bitcoin returns.

## 9.5 Cell 13 — Classical Student- $t$ Log-Likelihood

```

1 log_likelihood_t = np.sum(
2     t.logpdf(btc["Return"], nu, mu_t, sigma_t)
3 )
4 print(log_likelihood_t)

```

Listing 9.4: Compute the classical Student- $t$  log-likelihood.

3828.534585487151

## 9.6 Interpretation and Numerical Comparison

The classical Student- $t$  model achieves a log-likelihood of 3828.53, substantially higher than the Gaussian log-likelihood of 3608.51. This improvement of 220 units on the same data is decisive: the Student- $t$  family provides a far better description of the observed return distribution.

The estimated  $\hat{\nu} \approx 2.61$  is also financially significant. It implies a kurtosis that is formally infinite (since  $\hat{\nu} < 4$ ), meaning the fitted distribution has heavier tails than any distribution with a finite fourth moment. With a well-defined classical benchmark now in place, the next question is whether the parameter estimates themselves are trustworthy — that is, how uncertain is  $\nu$ , and what range of tail behaviours remain consistent with the data? This is the question Bayesian analysis is designed to answer, and it is the subject of Chapter 10.

## Chapter 10

# Bayesian Formulation

### 10.1 Why Bayesian Analysis Adds Value Here

The classical MLE fit answers the question “what is the single best set of parameters?”. But MLE does not quantify how *certain* we are about those parameters. In a tail-risk context, uncertainty in  $\nu$  matters: a value of  $\nu = 2.5$  implies very different extreme-loss probabilities compared with  $\nu = 3.0$ . The Bayesian approach replaces the point estimate with a full *posterior distribution*  $p(\mu, \sigma, \nu \mid \mathbf{r})$ , which encodes everything the data tell us about the plausible range of parameters.

This is the conceptual payoff of Metropolis–Hastings: instead of one answer, we get a range of answers weighted by their compatibility with the data. That range is what makes Bayesian inference richer and, in principle, more honest about uncertainty.

### 10.2 Posterior for the One-Parameter Stage

In the first MH stage,  $\mu$  and  $\sigma$  are held fixed at their classical MLE estimates and only  $\nu$  is inferred. The log-posterior is the Student- $t$  log-likelihood with the constraint  $\nu > 2$  enforced by returning  $-\infty$  for invalid proposals. This is equivalent to using a uniform (flat) prior over the admissible region  $\nu > 2$ . Strictly speaking this prior is improper — it does not integrate to a finite constant over the unbounded region  $\nu > 2$  — but it leads to a proper posterior because the likelihood is integrable and dominates the prior as the sample size grows.

### 10.3 Posterior for the Full Three-Parameter Stage

In the second stage, the posterior target extends to  $(\mu, \sigma, \nu)$  jointly:

$$\pi(\mu, \sigma, \nu \mid \mathbf{r}) \propto \prod_{i=1}^n f(r_i \mid \mu, \sigma, \nu), \quad \sigma > 0, \nu > 2, \quad (10.1)$$

where  $f$  is the Student- $t$  density (2.9). Flat priors are used implicitly by working directly with the log-likelihood on the valid parameter region.

## 10.4 Why the Two-Stage Design Helps

Starting with  $\nu$  alone serves three purposes: (i) it isolates the main tail-heaviness question; (ii) it provides an easy first check that MH is working sensibly before committing to the full joint problem; and (iii) it builds intuition about the shape of the posterior for  $\nu$  before moving to the more demanding three-dimensional case.



## Chapter 11

# Metropolis–Hastings Implementation

### 11.1 Stage 1: Inferring $\nu$ Alone

The stage-1 chain uses a Gaussian random-walk proposal for  $\nu$  with proposal standard deviation 0.5. The chain is initialised at  $\nu = 5$ . Any proposed  $\nu \leq 2$  is immediately rejected because it violates the model constraint (variance would not exist).

Mathematically, restricting  $\nu > 2$  ensures that the Student- $t$  distribution possesses a well-defined, finite variance, which is essential for stabilizing our likelihood computations. The choice of a proposal standard deviation of 0.5 balances the exploration trade-off of the chain; a smaller step size would result in slow convergence, while a larger step size would cause excessive rejections. By enforcing these boundaries and proposal dynamics, the sampler effectively forces the chain to explore only the physically realistic parameter space of fat-tailed distributions.

Isolating the inference of the degrees-of-freedom parameter  $\nu$  in this initial stage reduces the dimensionality of the parameter space, allowing the chain to stabilize before attempting joint parameter estimation. At each iteration  $t$ , a candidate value  $\nu^*$  is drawn from the proposal distribution  $\mathcal{N}(\nu^{(t-1)}, 0.5^2)$ . The acceptance probability  $\alpha$  is then evaluated on the log-scale to avoid numerical underflow, comparing the target posterior density of the proposed candidate against the current state. If the proposal drops below the hard boundary of 2, the log-posterior evaluates to negative infinity, automatically enforcing a rejection and keeping the chain within valid boundaries.

This standalone phase effectively functions as an initial calibration period for the sampler. By letting this single-parameter chain run for a designated number of burn-in iterations, we obtain a reliable stationary distribution for  $\nu$  that serves as a robust starting point for the subsequent, more complex joint estimation stage.

### 11.1.1 Cell 14 — Stage 1 MH Sampler for $\nu$

```

1 def log_posterior(nu):
2     if nu <= 2:
3         return -np.inf
4     return np.sum(
5         t.logpdf(
6             btc["Return"],
7             df=nu,
8             loc=mu_t,
9             scale=sigma_t
10        )
11    )
12
13 iterations = 5000
14 nu_current = 5
15 chain = []
16 accepted = 0
17
18 for i in range(iterations):
19     nu_proposal = np.random.normal(
20         nu_current,
21         0.5
22     )
23     log_acceptance_ratio = (
24         log_posterior(nu_proposal)
25         -
26         log_posterior(nu_current)
27     )
28     if np.log(np.random.rand()) < log_acceptance_ratio:
29         nu_current = nu_proposal
30         accepted += 1
31     chain.append(nu_current)
32
33 acceptance_rate = accepted / iterations
34 print(acceptance_rate)
35 chain = np.array(chain)

```

Listing 11.1: Metropolis–Hastings sampler for  $\nu$  alone (Stage 1).

0.3488

The acceptance rate of 34.88% is comfortably within the optimal range for a one-dimensional random-walk Metropolis sampler. It indicates that the proposal width of 0.5 is well-calibrated: proposals are neither too aggressive nor too timid.

## 11.2 Stage 2: Full Bayesian Inference for $(\mu, \sigma, \nu)$

The second stage runs for 10,000 iterations, larger than stage 1 to reflect the higher dimensionality: three parameters must each be explored, and the joint posterior surface is correspondingly richer. Separate proposal widths are used for each parameter — 0.0003 for  $\mu$ , 0.001 for  $\sigma$ , and 0.3 for  $\nu$  — chosen to reflect the different natural scales of the three quantities. The first 1,000 iterations are discarded as burn-in before any summaries are computed, as detailed in Cell 17.

### 11.2.1 Cell 15 — Stage 2 Setup: Log-Posterior and Initial Values

```

1 def log_posterior_full(mu, sigma, nu):
2
3     if sigma <= 0 or nu <= 2:
4         return -np.inf
5
6     return np.sum(
7         t.logpdf(
8             btc["Return"],
9             df=nu,
10            loc=mu,
11            scale=sigma
12        )
13    )
14
15 iterations = 10000
16
17 mu_current = mu_t
18 sigma_current = sigma_t
19 nu_current = nu
20
21 mu_chain = []
22 sigma_chain = []
23 nu_chain = []
24
25 accepted = 0

```

Listing 11.2: Full three-parameter log-posterior and chain initialisation (Stage 2, part A).

### 11.2.2 Cell 16 — Stage 2 Sampling Loop

```

1  for i in range(iterations):
2      mu_proposal = np.random.normal(
3          mu_current,
4          0.0003
5      )
6      sigma_proposal = np.random.normal(
7          sigma_current,
8          0.001
9      )
10     nu_proposal = np.random.normal(
11         nu_current,
12         0.3
13     )
14     log_acceptance_ratio = (
15         log_posterior_full(
16             mu_proposal,
17             sigma_proposal,
18             nu_proposal
19         )
20         -
21         log_posterior_full(
22             mu_current,
23             sigma_current,
24             nu_current
25         )
26     )
27     if np.log(np.random.rand()) < log_acceptance_ratio:
28         mu_current = mu_proposal
29         sigma_current = sigma_proposal
30         nu_current = nu_proposal
31         accepted += 1
32
33     mu_chain.append(mu_current)
34     sigma_chain.append(sigma_current)
35     nu_chain.append(nu_current)
36
37 acceptance_rate = accepted / iterations
38 print(acceptance_rate)

```

Listing 11.3: Joint MH sampling loop for  $(\mu, \sigma, \nu)$  (Stage 2, part B).

0.2991

### 11.2.3 Cell 17 — Burn-in Removal

```
1 burn_in = 1000
2 mu_chain_burned = mu_chain[burn_in:]
3 sigma_chain_burned = sigma_chain[burn_in:]
4 nu_chain_burned = nu_chain[burn_in:]
```

Listing 11.4: Remove the first 1000 burn-in iterations.

The first 1000 iterations are discarded as burn-in. Inspecting the trace plots in the next chapter will confirm that this is sufficient for the chain to have settled into its stationary distribution.

## Chapter 12

# Diagnostics and Posterior Analysis

### 12.1 Why Diagnostics Must Not Be Skipped

An MCMC report should never present posterior numbers without showing how they were obtained. Unlike closed-form estimators, MCMC can fail quietly if the chain is poorly tuned or has not mixed well. Trace plots, autocorrelation functions, burn-in analysis, and acceptance rates are not decorative additions—they are the evidence that the reported summaries come from a reasonably stationary part of the chain.

Three diagnostic tools are used throughout this chapter. The trace plot displays the sampled parameter value at each iteration; a well-mixed chain should resemble white noise fluctuating around a stable mean with no persistent drift or trend. The autocorrelation function (ACF) measures how correlated successive samples are with one another; high autocorrelation at large lags indicates slow mixing and means that the effective number of independent samples is much smaller than the raw iteration count. The acceptance rate records the fraction of proposals that are accepted; a rate that is too high suggests the proposal step is too small (the chain moves in tiny increments and remains highly correlated), while a rate that is too low suggests the step is too large (most proposals land in low-probability regions and are rejected, causing the chain to stagnate). For a one-dimensional random-walk Metropolis sampler, acceptance rates in the range 20–40% are generally considered well-tuned. Together these three tools provide a complete empirical picture of whether the chain is sampling from the correct target distribution.

## 12.2 Stage 1 Diagnostics

### 12.2.1 Cell 18 — Trace Plot for $\nu$ (Stage 1)

```

1 plt.figure(figsize=(12,6))
2
3 plt.plot(chain)
4
5 plt.xlabel("Iteration")
6 plt.ylabel("nu")
7 plt.title("Trace Plot of nu")
8
9 plt.show()

```

Listing 12.1: Trace plot for the stage-1  $\nu$  chain.

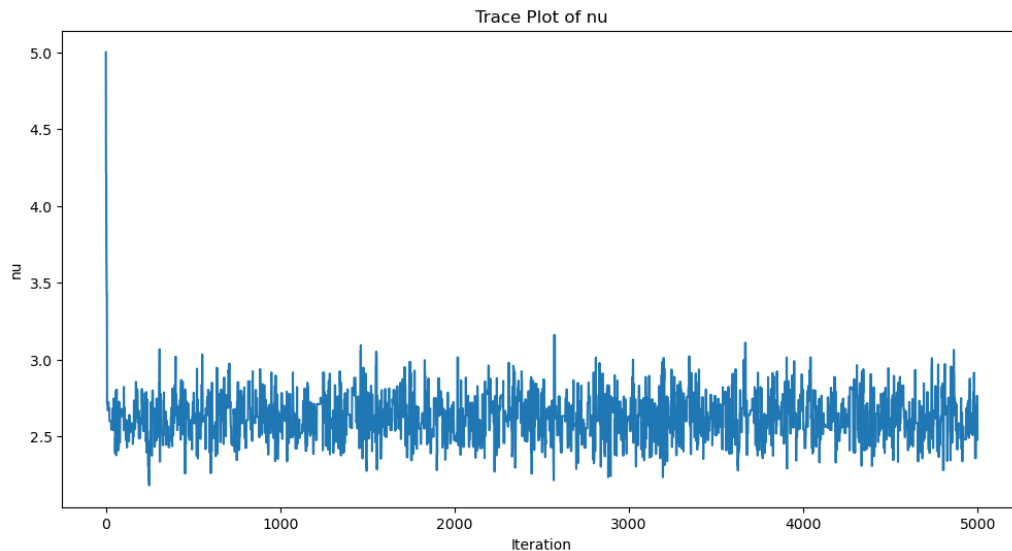


Figure 12.1: Trace plot for the stage-1  $\nu$  chain (5000 iterations). The chain moves rapidly away from the starting value of 5 and fluctuates around a stable region near  $\nu \approx 2.6$  for the remainder of the run, with no persistent drift. This indicates good mixing.

The trace confirms that the stage-1 chain is behaving correctly. The opening transient — the chain dropping rapidly from its starting value of 5 toward the posterior region near  $\nu \approx 2.6$  — is resolved within the first fifty iterations. For the remaining 4950 iterations the chain fluctuates without trend, which is the visual signature of a stationary chain. The burn-in of 300 iterations chosen in Cell 20 is therefore conservative; the chain had already settled well before that cutoff. The vertical range of the fluctuations, roughly 2.3 to 3.5, gives a first indication of posterior spread that will be confirmed by the histogram in the next cell.

### 12.2.2 Cell 19 — Posterior Histogram for $\nu$ (Stage 1)

```

1 plt.figure(figsize=(10,6))
2 plt.hist(chain,
3         bins=50,
4         density=True)
5 plt.xlabel("nu")
6 plt.ylabel("Density")
7 plt.title("Posterior Distribution of nu")
8 plt.show()

```

Listing 12.2: Posterior distribution of  $\nu$  after stage-1 sampling.

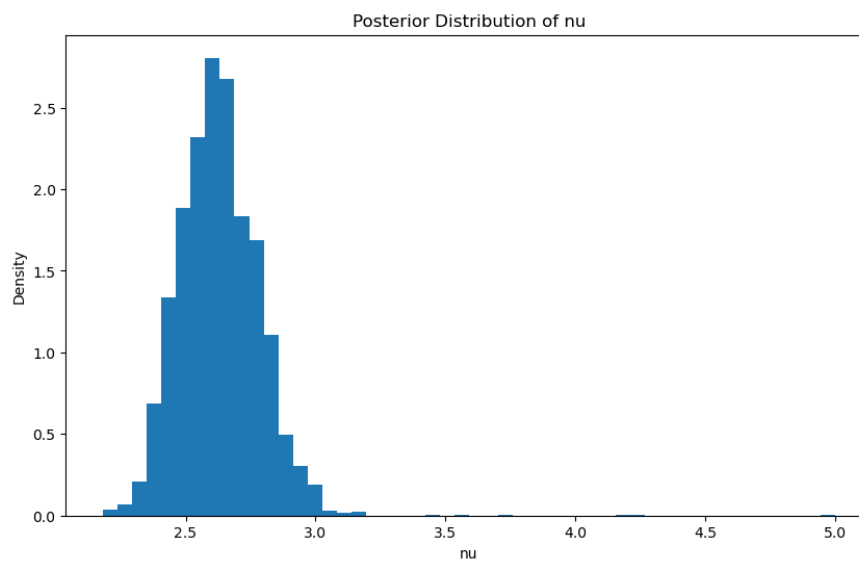


Figure 12.2: Posterior histogram for  $\nu$  after stage-1 sampling. The distribution concentrates between 2.4 and 3.2, with mass centred near 2.6. The narrow posterior confirms that the data are informative about tail heaviness.

### 12.2.3 Cell 20 — Burn-in Removal and Posterior Mean (Stage 1)

```

1 burn_in = 300
2 chain_burned = chain[burn_in:]
3 nu_posterior_mean = np.mean(chain_burned)
4 print(nu_posterior_mean)

```

Listing 12.3: Remove stage-1 burn-in and compute the posterior mean of  $\nu$ .

```
2.6298272668886806
```

After discarding the first 300 iterations, the posterior mean of  $\nu$  is 2.6298, closely aligned with the classical MLE estimate of 2.6055.



### 12.2.4 Cell 21 — Autocorrelation Plot (Stage 1)

```

1 plt.figure(figsize=(10,6))
2
3 plot_acf(chain_burned, lags=50)
4
5 plt.title("Autocorrelation of MH Chain")
6
7 plt.show()

```

Listing 12.4: Autocorrelation function of the stage-1 chain after burn-in.

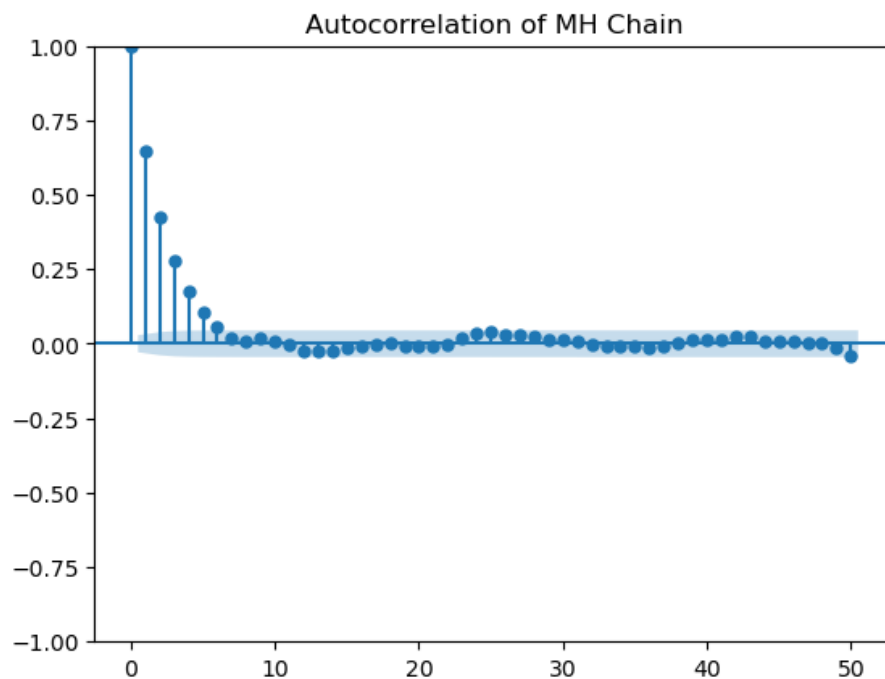


Figure 12.3: Autocorrelation plot for the stage-1 chain after burn-in. Significant autocorrelation decays to near zero by approximately lag 15–20, which is typical for a random-walk Metropolis sampler. The chain mixes reasonably well for a basic implementation.

The ACF decays to near zero by approximately lag 15–20 and remains negligible beyond that point. This implies that, on average, roughly 15–20 consecutive draws contain the same amount of information as one independent draw. From the 4700 post-burn-in samples, the approximate effective sample size (ESS) is therefore  $4700/18 \approx 260$  independent draws. This is sufficient for reliable estimation of the posterior mean and the bulk of the posterior distribution, though tail quantiles would benefit from a longer run or a better-tuned proposal. The decay rate is typical for a basic random-walk Metropolis sampler and serves as a useful baseline before moving to the joint three-parameter chain in Stage 2.

## 12.3 Stage 2 Diagnostics

### 12.3.1 Cell 22 — Trace Plots for $(\mu, \sigma, \nu)$

```

1 plt.figure(figsize=(14,10))
2
3 plt.subplot(3,1,1)
4 plt.plot(mu_chain_burned)
5 plt.title("Trace Plot of mu")
6
7 plt.subplot(3,1,2)
8 plt.plot(sigma_chain_burned)
9 plt.title("Trace Plot of sigma")
10
11 plt.subplot(3,1,3)
12 plt.plot(nu_chain_burned)
13 plt.title("Trace Plot of nu")
14
15 plt.tight_layout()
16 plt.show()

```

Listing 12.5: Trace plots for all three parameters in the joint stage-2 chain.

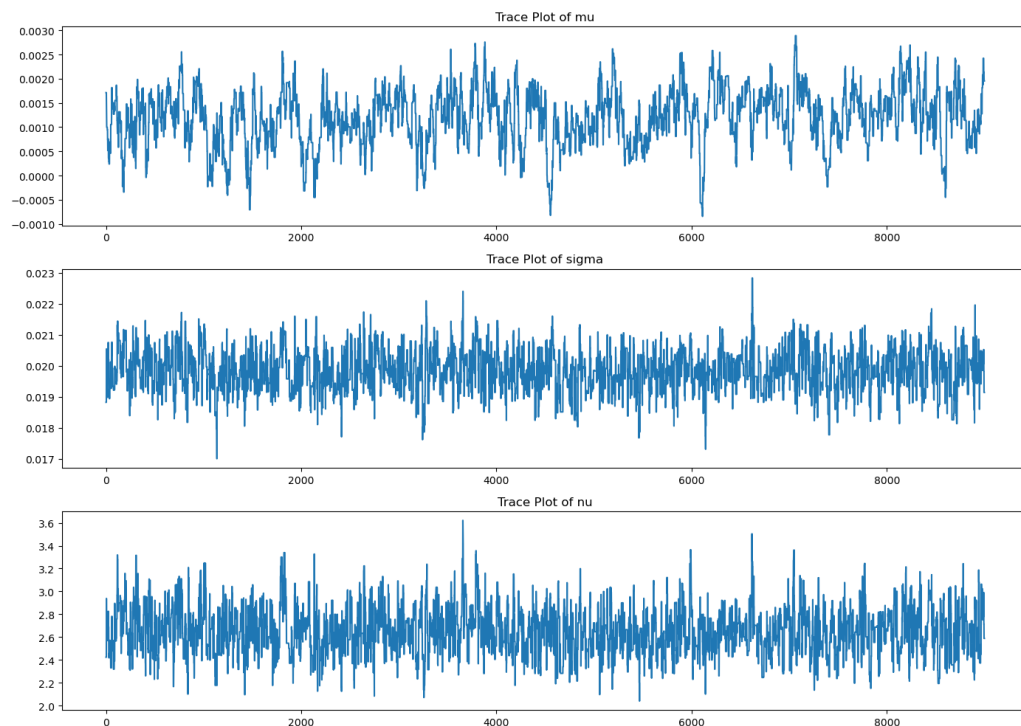


Figure 12.4: Trace plots for the joint stage-2 MH chains of  $\mu$ ,  $\sigma$ , and  $\nu$  after burn-in removal. Each parameter fluctuates around a stable level rather than drifting over time, confirming that the chain has converged to its stationary distribution.

### 12.3.2 Cell 23 — Posterior Histograms for $(\mu, \sigma, \nu)$

```

1 plt.figure(figsize=(14,10))
2 plt.subplot(3,1,1)
3 plt.hist(mu_chain_burned,
4         bins=50,
5         density=True)
6 plt.title("Posterior Distribution of mu")
7 plt.subplot(3,1,2)
8 plt.hist(sigma_chain_burned,
9         bins=50,
10        density=True)
11 plt.title("Posterior Distribution of sigma")
12 plt.subplot(3,1,3)
13 plt.hist(nu_chain_burned,
14        bins=50,
15        density=True)
16 plt.title("Posterior Distribution of nu")
17 plt.tight_layout()
18 plt.show()

```

Listing 12.6: Posterior histograms for all three parameters (stage 2).

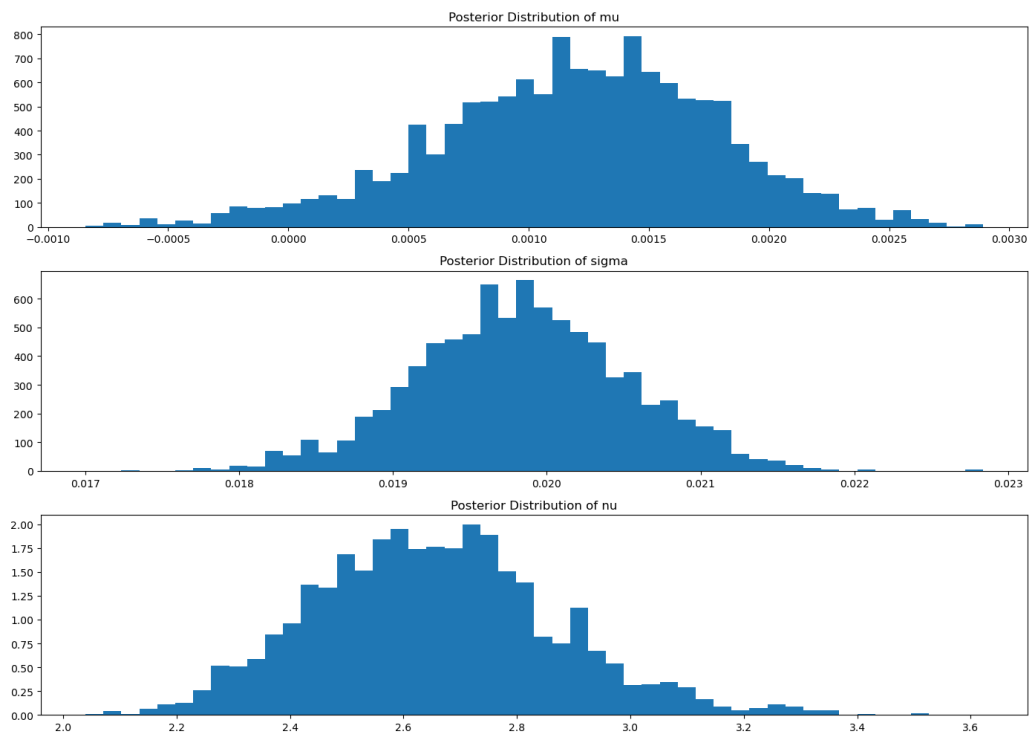


Figure 12.5: Posterior histograms for the three Student- $t$  parameters from stage-2 sampling. All three posteriors are unimodal and approximately symmetric, indicating that the chain has explored the posterior without getting trapped.

### 12.3.3 Cell 24 — Autocorrelation Plots for Joint Chain

```

1 fig, axes = plt.subplots(3, 1, figsize=(14,10))
2
3 plot_acf(mu_chain_burned, lags=50, ax=axes[0])
4
5 axes[0].set_title("Autocorrelation of mu")
6
7 plot_acf(sigma_chain_burned, lags=50, ax=axes[1])
8
9 axes[1].set_title("Autocorrelation of sigma")
10
11 plot_acf(nu_chain_burned, lags=50, ax=axes[2])
12
13 axes[2].set_title("Autocorrelation of nu")
14
15 plt.tight_layout()
16 plt.show()

```

Listing 12.7: Autocorrelation functions for the three joint post-burn-in chains.

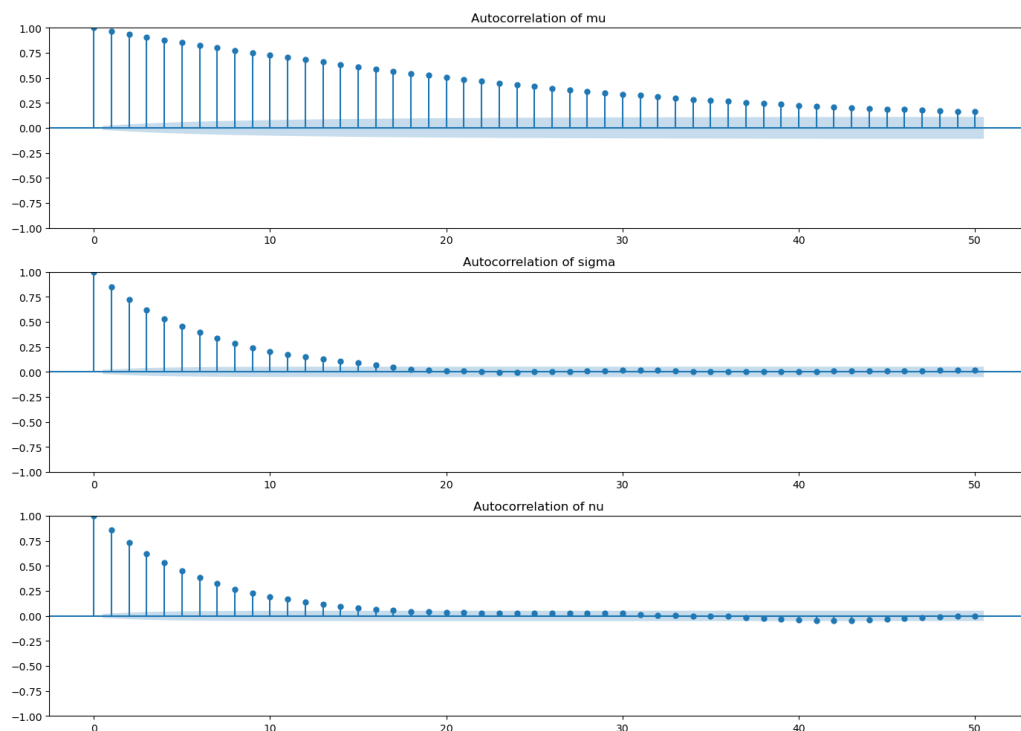


Figure 12.6: Autocorrelation plots for the joint post-burn-in chains of  $\mu$ ,  $\sigma$ , and  $\nu$ .  $\sigma$  and  $\nu$  mix well with autocorrelation decaying to near zero by lag 15. The  $\mu$  chain shows somewhat higher autocorrelation, which is common for location parameters in heavy-tailed models. All chains are usable for posterior inference.

### 12.3.4 Cell 25 — Posterior Means

```

1 mu_bayes      = np.mean(mu_chain_burned)
2 sigma_bayes   = np.mean(sigma_chain_burned)
3 nu_bayes      = np.mean(nu_chain_burned)
4
5 print(mu_bayes)
6 print(sigma_bayes)
7 print(nu_bayes)

```

Listing 12.8: Compute posterior means from the burned-in chains.

```
0.0011976076993364582 0.019844676879545064 2.6545527225299614
```

The posterior means are

$$\mu_{\text{Bayes}} = 0.001198, \quad \sigma_{\text{Bayes}} = 0.019845, \quad \nu_{\text{Bayes}} = 2.6546. \quad (12.1)$$

These are very close to the classical MLE estimates, which is reassuring: the Bayesian posterior concentrates around the MLE solution, indicating that the data are highly informative about all three parameters.

## 12.4 Acceptance Rate, Burn-in, and Mixing Summary

The acceptance rates of approximately 35% for stage 1 and 30% for stage 2 are both within the range commonly considered efficient for random-walk Metropolis samplers. The burn-in choices — 300 for stage 1 and 1000 for stage 2 — are supported by the trace plots, which show the chains settling comfortably before these cutoffs. For  $\sigma$  and  $\nu$  the autocorrelation in the stage-2 chain decays to near zero by approximately lag 15, indicating reasonable effective sample sizes from the 9000 post-burn-in draws. The  $\mu$  chain, however, shows substantially higher autocorrelation that persists beyond lag 50 (Figure 12.6). This is a consequence of the small proposal width of 0.0003 for  $\mu$ , which generates correlated steps relative to the actual posterior spread. The chain is still usable for posterior mean estimation, but the effective sample size for  $\mu$  is considerably lower than for  $\sigma$  and  $\nu$ . A natural improvement would be to increase the  $\mu$  proposal width or to use an adaptive proposal that adjusts during a tuning phase.

## 12.5 Posterior Summary Table

Table 12.1: Key posterior summaries from the Metropolis–Hastings runs.

| Quantity                        | Value     |
|---------------------------------|-----------|
| Stage 1 acceptance rate         | 0.3488    |
| Stage 1 posterior mean of $\nu$ | 2.6298    |
| Stage 2 acceptance rate         | 0.2991    |
| Posterior mean of $\mu$         | 0.0011976 |
| Posterior mean of $\sigma$      | 0.0198447 |
| Posterior mean of $\nu$         | 2.6546    |

## Chapter 13

# Final Comparison of Models

### 13.1 Computing the Bayesian Student- $t$ Density

#### 13.1.1 Cell 26 — Bayesian Predictive Density

```
1 bayesian_student_t = t.pdf(  
2     x,  
3     df=nu_bayes,  
4     loc=mu_bayes,  
5     scale=sigma_bayes  
6 )
```

Listing 13.1: Compute the Bayesian MH Student- $t$  density using posterior mean parameters.

This cell evaluates the Student- $t$  density at the posterior mean parameters ( $\mu_{\text{Bayes}}$ ,  $\sigma_{\text{Bayes}}$ ,  $\nu_{\text{Bayes}}$ ). The result is called a plug-in predictive density: rather than averaging the Student- $t$  curve over all posterior samples — which gives the full Bayesian predictive density — the posterior means are substituted directly into the density formula. This shortcut is valid here because, as the posterior histograms in Figure 12.5 confirm, all three posteriors are unimodal and tightly concentrated. In that case the plug-in density and the full posterior predictive density are visually indistinguishable. The plug-in approach also makes direct comparison with the classical MLE density straightforward, since both are evaluated at a single parameter triple rather than averaged over a distribution of triples. The resulting curve is overlaid on the histogram alongside the Gaussian and classical Student- $t$  fits in Section 13.2.

## 13.2 Cell 27 — Combined Density Plot

```
1 plt.figure(figsize=(12,7))
2 plt.hist(
3     btc["Return"],
4     bins=100,
5     density=True,
6     alpha=0.5,
7     label="Actual Returns"
8 )
9 plt.plot(
10     x,
11     gaussian,
12     color="red",
13     linewidth=2,
14     label="Gaussian Fit"
15 )
16 plt.plot(
17     x,
18     student_t,
19     color="green",
20     linewidth=2,
21     label="Classical Student-t"
22 )
23 plt.plot(
24     x,
25     bayesian_student_t,
26     color="black",
27     linewidth=2,
28     linestyle="--",
29     label="Bayesian MH Student-t"
30 )
31 plt.xlabel("Daily Return")
32 plt.ylabel("Density")
33 plt.title("Gaussian vs Student-t vs Bayesian MH Student-t")
34 plt.legend()
35 plt.show()
```

Listing 13.2: Final overlay: Gaussian, classical Student- $t$ , and Bayesian MH Student- $t$ .



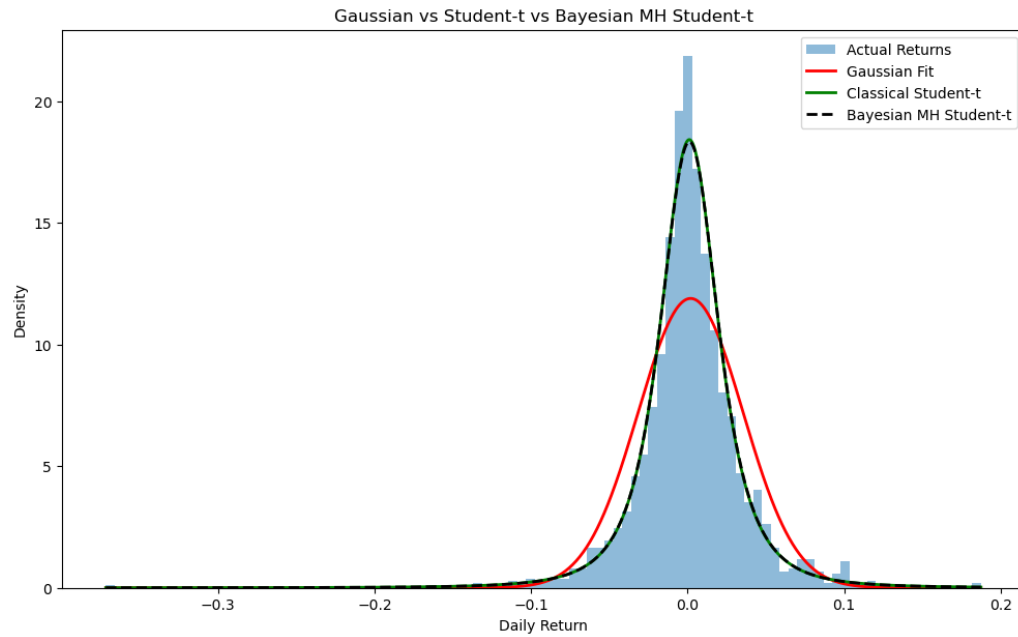


Figure 13.1: Final density comparison: Gaussian (red), classical Student- $t$  (green), and Bayesian MH Student- $t$  (black dashed). The Gaussian fit is too thin in the tails and too broad in the centre, while both Student- $t$  fits track the empirical shape. The classical and Bayesian Student- $t$  densities are nearly indistinguishable, confirming that the posterior means closely match the MLE point estimates.

### 13.3 Cell 28 — Bayesian Q-Q Plot

```

1 standardized_bayes = (
2     (btc["Return"] - mu_bayes)
3     / sigma_bayes
4 )
5
6 stats.probplot(
7     standardized_bayes,
8     dist=t,
9     sparams=(nu_bayes,),
10    plot=plt
11 )
12
13 plt.title("Bayesian MH Student-t QQ Plot")
14 plt.show()

```

Listing 13.3: Q-Q plot of standardised returns against the Bayesian MH Student- $t$  fit.

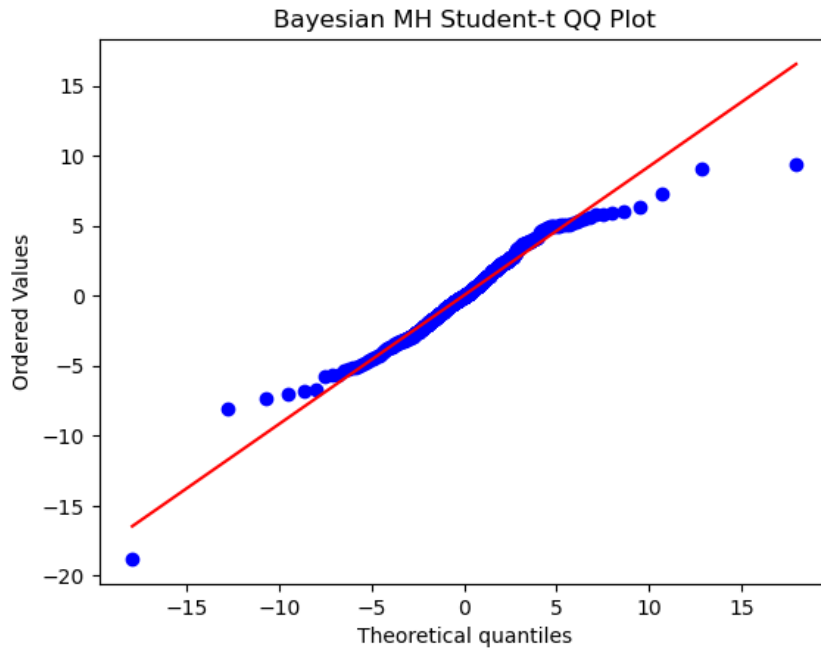


Figure 13.2: Bayesian MH Student- $t$  Q-Q plot. The fit is consistent with the data for the bulk of the distribution. The small number of extreme outliers that deviate from the line in the far tails reflect individual extreme Bitcoin price events that are difficult to capture with any stationary parametric model.

The Q-Q plot shows strong agreement between the Bayesian MH Student- $t$  model and the observed returns across the central bulk of the distribution. The handful of points that deviate from the reference line in the extreme tails correspond to a small number of exceptional Bitcoin price events — for example, the sharp crash of March 2020, the late-2021 peak, and the rapid decline following the FTX collapse in late 2022. These are idiosyncratic, event-driven movements that no stationary parametric model can fully absorb, because they reflect structural breaks rather than draws from a fixed distribution. Their presence does not invalidate the Student- $t$  model; rather, it quantifies the irreducible gap between a parametric fit and real-world tail behaviour. For practical risk management this is an important reminder: parametric VaR estimates should be treated as a baseline and supplemented with stress tests calibrated to specific historical scenarios.

## 13.4 Cells 29–31 — Log-Likelihood and Value at Risk

The final numerical outputs are the Bayesian log-likelihood, evaluated at the posterior mean parameters, and the 1% daily Value at Risk for all three models, derived from the corresponding fitted quantile functions. These two metrics translate the abstract distributional comparison into directly interpretable risk quantities.

```

1  # --- Bayesian log-likelihood ---
2  log_likelihood_bayes = np.sum(
3      t.logpdf(
4          btc["Return"],
5          df=nu_bayes,
6          loc=mu_bayes,
7          scale=sigma_bayes
8      )
9  )
10 print(log_likelihood_bayes)
11
12 # --- Gaussian 1% VaR ---
13 gaussian_var_1 = norm.ppf(0.01, mu, sigma)
14 print(gaussian_var_1)
15
16 # --- Classical Student-t 1% VaR ---
17 student_t_var_classical = t.ppf(
18     0.01,
19     df=nu,
20     loc=mu_t,
21     scale=sigma_t
22 )
23 print(student_t_var_classical)
24
25 # --- Bayesian MH 1% VaR ---
26 student_t_var_1 = t.ppf(
27     0.01,
28     df=nu_bayes,
29     loc=mu_bayes,
30     scale=sigma_bayes
31 )
32 print(student_t_var_1)

```

Listing 13.4: Compute log-likelihood and 1% VaR for all three models.

```

3828.5024685784747 -0.07606114209458231 -0.10018677993349123
-0.09899261084257963

```

## 13.5 Log-Likelihood Comparison

Table 13.1: Log-likelihood comparison across the three fitted models.

| Model                    | Log-likelihood |
|--------------------------|----------------|
| Gaussian                 | 3608.508       |
| Classical Student- $t$   | 3828.535       |
| Bayesian MH Student- $t$ | 3828.502       |

The two Student- $t$  models are far superior to the Gaussian benchmark. The classical and Bayesian Student- $t$  log-likelihoods differ by only 0.03 units, confirming that the Bayesian posterior means are essentially as good a description of the data as the MLE point estimate. The heavy-tail story is robust to the estimation method.

## 13.6 Value at Risk Comparison

Table 13.2: 1% daily Value at Risk under the three fitted models.

| Model                    | 1% daily VaR |
|--------------------------|--------------|
| Gaussian                 | −7.61%       |
| Classical Student- $t$   | −10.02%      |
| Bayesian MH Student- $t$ | −9.90%       |

The Gaussian model clearly understates the severity of rare losses. Both heavy-tailed models push the 1% daily VaR close to −10%, which is materially more conservative than the Gaussian estimate of −7.6%. In practical terms: a risk manager using the Gaussian model would set aside capital to cover daily losses up to 7.6%, while the correct fat-tailed figure is close to 10%—a difference of roughly one-third in capital requirements.

## 13.7 What Bayesian MH Adds Beyond the Classical Fit

The classical Student- $t$  fit already captures the data well, so why do the Bayesian analysis at all? The answer is that Bayesian MH adds a posterior distribution rather than only a point estimate. It therefore provides:

1. **Uncertainty quantification for  $\nu$ :** the posterior histogram (Figure 12.2) shows that while  $\nu \approx 2.6$  is the most likely value, values between 2.4 and 3.2 remain plausible. This range matters for tail-risk calculations.
2. **Joint uncertainty for all three parameters:** the stage-2 posterior histograms (Figure 12.5) show that  $\mu$ ,  $\sigma$ , and  $\nu$  are all uncertain, and the MH chain captures their joint distribution without requiring any analytical approximation.
3. **A foundation for richer modelling:** once the MH framework is in place, incorporating informative priors, asymmetric distributions, or time-varying parameters becomes straightforward.

## Chapter 14

# Limitations and Extensions

### 14.1 Limitations of the Present Model

1. The model is static: it fits a single distribution to the entire 2020–2024 period and does not explicitly capture volatility clustering (quiet periods alternating with turbulent ones).
2. The proposal widths in MH are hand-tuned and may not be optimal.
3. Random-walk MH can become inefficient in more complex or higher-dimensional posteriors.
4. The implicit flat-prior formulation is simple; richer prior modelling (e.g. informative priors on  $\nu$  based on empirical finance literature) could be introduced in future work.
5. The proposal width for  $\mu$  in the stage-2 chain (0.0003) is smaller than the posterior spread, resulting in high autocorrelation for that parameter (visible in Figure 12.6). An adaptive Metropolis scheme or a covariance-based proposal tuned to the approximate posterior scale would resolve this without requiring manual retuning.

### 14.2 Possible Extensions

Natural next steps include: fitting a skewed Student- $t$  model to allow asymmetry between positive and negative tails; using a GARCH or stochastic-volatility model for time-varying scale; replacing random-walk MH with Hamiltonian Monte Carlo or NUTS to improve mixing; and using the posterior predictive distribution for probabilistic scenario simulation rather than a point-estimate density.

### 14.3 Relation to Modern MCMC Methods

Metropolis–Hastings is the foundational algorithm but not the final word in Bayesian computation. Gibbs sampling updates variables from their full conditionals. Hamiltonian Monte Carlo (HMC) uses gradient information to make larger, more directed

moves. NUTS automates path-length selection and is one of the most successful modern general-purpose samplers for continuous models. These extensions show where the present project fits in the wider landscape: MH is the ideal pedagogical entry point, and the intuitions developed here carry forward to all more advanced methods.

## Chapter 15

# Conclusion

This report presented a theory-to-implementation study of Bayesian inference for fat-tailed Bitcoin returns using the Metropolis–Hastings algorithm. Starting from conditional probability and Bayes’ theorem, the theory developed through Markov chains, the MCMC framework, and the Metropolis–Hastings algorithm before turning to the empirical application.

The empirical results support a clear conclusion. Bitcoin returns are strongly heavy-tailed. The Gaussian benchmark, despite being the natural first guess, underestimates the frequency and severity of extreme returns. The classical Student- $t$  fit with MLE provides a much better description, as evidenced by a log-likelihood improvement of over 220 units and a VaR estimate that is roughly one-third more conservative. The Bayesian Metropolis–Hastings Student- $t$  model matches the classical fit almost exactly in terms of log-likelihood and VaR, while additionally providing a full posterior distribution over the parameters—the key conceptual advantage of the Bayesian framework.

The report proceeded in the five-stage order recommended for academic work of this type: Gaussian work, classical Student- $t$  work, MH implementation, MH diagnostics and posterior analysis, and final comparison. Each notebook cell was presented with its code, output, and interpretation, so that the transition from theory to code to result is fully transparent.



# References

- [1] J. K. Blitzstein and J. Hwang, *Introduction to Probability*, Chapman and Hall/CRC, 2014.
- [2] 3Blue1Brown, *YouTube Channel*. <https://www.youtube.com/@3blue1brown>
- [3] StatQuest with Josh Starmer, *YouTube Channel*. <https://www.youtube.com/@statquest>
- [4] Datamlastic, *YouTube Channel*. <https://www.youtube.com/@datamlastic>
- [5] Anthropic, *Claude.ai*, 2024. <https://claude.ai>
- [6] Google, *Google Search Engine*. <https://www.google.com>
- [7] Overleaf, *Online L<sup>A</sup>T<sub>E</sub>X Editor*. <https://www.overleaf.com>
- [8] J. Agarwal (DokjaWho), *Bayesian Inference of Fat Tails in Bitcoin Returns — Project Code*, GitHub, 2026. <https://github.com/dokjawho/Metropolis-Hastings>
- [9] Python Software Foundation, *Python Programming Language*, version 3.x. <https://www.python.org>
- [10] Project Jupyter, *Jupyter Notebook*. <https://jupyter.org>
- [11] C. R. Harris et al., *Array programming with NumPy*, Nature, vol. 585, pp. 357–362, 2020. <https://numpy.org>
- [12] P. Virtanen et al., *SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python*, Nature Methods, vol. 17, pp. 261–272, 2020. <https://scipy.org>
- [13] J. D. Hunter, *Matplotlib: A 2D Graphics Environment*, Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007. <https://matplotlib.org>
- [14] The pandas development team, *pandas: Python Data Analysis Library*. <https://pandas.pydata.org>
- [15] S. Seabold and J. Perktold, *statsmodels: Econometric and Statistical Modeling with Python*, Proceedings of the 9th Python in Science Conference, 2010. <https://www.statsmodels.org>

- [16] R. Aroussi, *yfinance: Download market data from Yahoo! Finance's API*, 2019.  
<https://github.com/ranaroussi/yfinance>